Practical No 2-A

Fragments and Activity Interaction

Aim: Develop an Android application to demonstrate Fragment lifecycle methods.

Fragment

A Fragment is a reusable portion of an Activity that represents a part of the user interface and can be combined with other fragments in a single activity.

Fragment Lifecycle Methods

Just like an Activity, a Fragment also has a lifecycle.

onAttach()
↓
onCreate()
↓
onCreateView()
↓
onViewCreated()
↓
onStart()
↓
onResume()

(App Running)

onPause()
↓
onStop()
↓
onDestroyView()
↓
onDestroy()
↓
onDetach()

Code:

1) **MainActivity.java (Keep as it is)**

```
package com.example.nepnewpract;

import android.os.Bundle;
```

```
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
   @Override
   protected void onCreate(Bundle savedInstanceState) {
       super.onCreate(savedInstanceState);
       EdgeToEdge.enable(this);
       setContentView(R.layout.activity_main);
       ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
insets) -> {
           Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
           v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
           return insets;
       });
   }

}
```

2) MyFragment.java

```
package com.example.twoa;

import android.content.Context;
import android.os.Bundle;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Toast;
```

```
/**
* A simple {@link Fragment} subclass.
* Use the {@link MyFragment#newInstance}
factory method to
* create an instance of this fragment.
*/
public class MyFragment extends Fragment {
    @Override
    public void onAttach(@NonNull Context
context) {
        super.onAttach(context);
        Toast.makeText(getActivity(),
"onAttach()", Toast.LENGTH_SHORT).show();
    }

    // TODO: Rename parameter arguments,
choose names that match
    // the fragment initialization
parameters, e.g. ARG_ITEM_NUMBER
    private static final String ARG_PARAM1 =
"param1";
    private static final String ARG_PARAM2 =
"param2";

    // TODO: Rename and change types of
parameters
    private String mParam1;
```

```
    private String mParam2;

    public MyFragment() {
        // Required empty public constructor
    }

    /**
     * Use this factory method to create a
new instance of
     * this fragment using the provided
parameters.
     *
     * @param param1 Parameter 1.
     * @param param2 Parameter 2.
     * @return A new instance of fragment
MyFragment.
     */
    // TODO: Rename and change types and
number of parameters
    public static MyFragment
newInstance(String param1, String param2) {
        MyFragment fragment = new
MyFragment();
        Bundle args = new Bundle();
        args.putString(ARG_PARAM1, param1);
        args.putString(ARG_PARAM2, param2);
        fragment.setArguments(args);
        return fragment;
```

```
    }

    @Override
    public void onCreate(Bundle
savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (getArguments() != null) {
            mParam1 =
getArguments().getString(ARG_PARAM1);
            mParam2 =
getArguments().getString(ARG_PARAM2);

        }
        Toast.makeText(getActivity(),
"onCreate", Toast.LENGTH_SHORT).show();
    }

    @Override
    public View onCreateView(LayoutInflater
inflater, ViewGroup container,
                             Bundle
savedInstanceState) {
        Toast.makeText(getActivity(),
"onCreateView()",
Toast.LENGTH_SHORT).show();

        // Inflate the layout for this
fragment
```

```
        return
inflater.inflate(R.layout.fragment_my,
container, false);
    }
    @Override
    public void onViewCreated(@NonNull View
view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view,
savedInstanceState);
        Toast.makeText(getActivity(),
"onViewCreated()",
Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onStart() {
        super.onStart();
        Toast.makeText(getActivity(),
"onStart()", Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onResume() {
        super.onResume();
        Toast.makeText(getActivity(),
                "onResume()",
                Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onPause() {
```

```
        super.onPause();
        Toast.makeText(getActivity(),
                "onPause()",
                Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onStop() {
        super.onStop();
        Toast.makeText(getActivity(),
                "onStop()",
                Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onDestroyView() {
        super.onDestroyView();
        Toast.makeText(getActivity(),
"onDestroyView()",
Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
        Toast.makeText(getActivity(),
                "onDestroy()",
                Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onDetach() {
```

```
        super.onDetach();
        Toast.makeText(getActivity(),
                "onDetach()",
                Toast.LENGTH_SHORT).show();
    }

}
```

3) activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity">

   <fragment
       android:id="@+id/myFragment"
      android:name="com.example.nepnewpract.MyFragment"
       android:layout_width="match_parent"
       android:layout_height="match_parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

4) fragment_my.xml

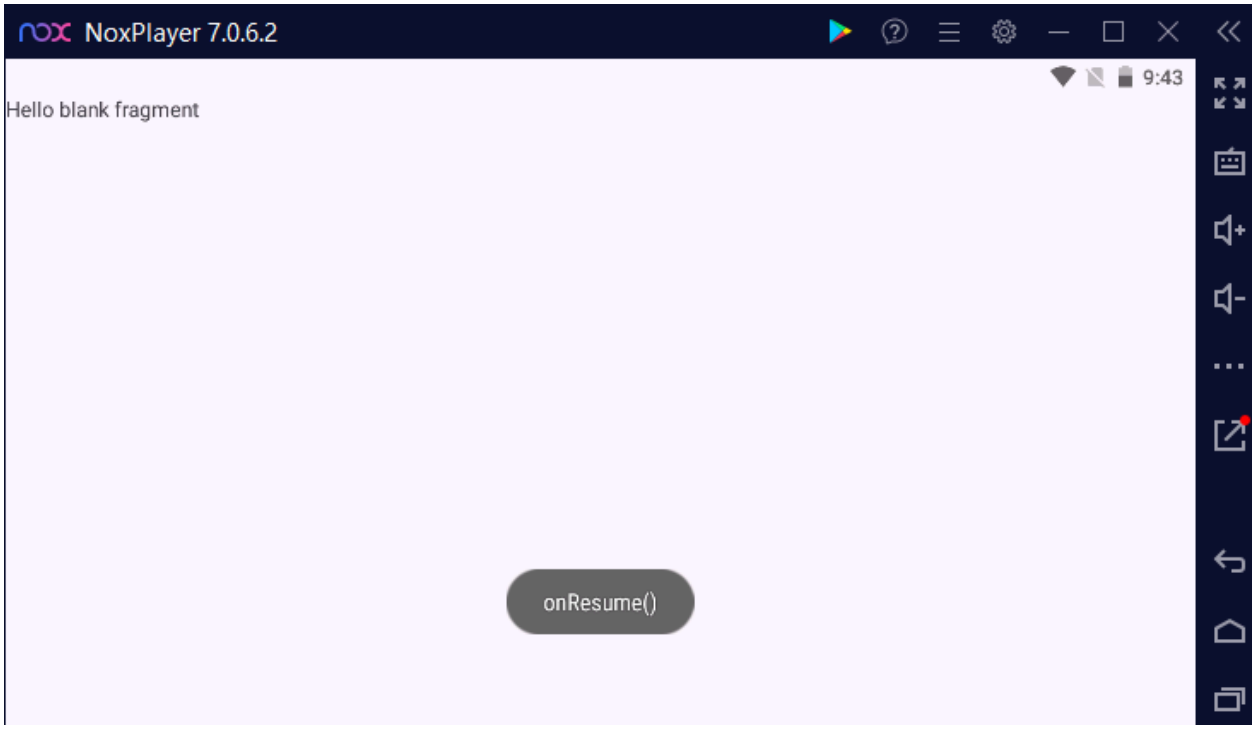
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MyFragment"
   >

   <!-- TODO: Update blank fragment layout -->
   <TextView
       android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="Fragment Lifecycle Demo"
        android:textSize="24sp"/>

</LinearLayout>
```

Output:



Practical No 3-B

Aim: Develop an Android application using RelativeLayout to align UI components dynamically.

RelativeLayout:

RelativeLayout is a ViewGroup that positions UI components relative to:

- Parent layout
- Other UI components

Properties of Relative Layout with respect to parent

- `android:layout_centerInParent`
- `android:layout_centerHorizontal`
- `android:layout_centerVertical`
- `android:layout_alignParentTop`
- `android:layout_alignParentRight`
- `android:layout_alignParentBottom`
- `android:layout_alignParentLeft`

With respect to other views

- `android:layout_alignTop`
- `android:layout_alignRight`
- `android:layout_alignBottom`
- `android:layout_alignLeft`
- `android:layout_above`
- `android:layout_below`
- `android:layout_toLeftOf`
- `android:layout_toRightOf`

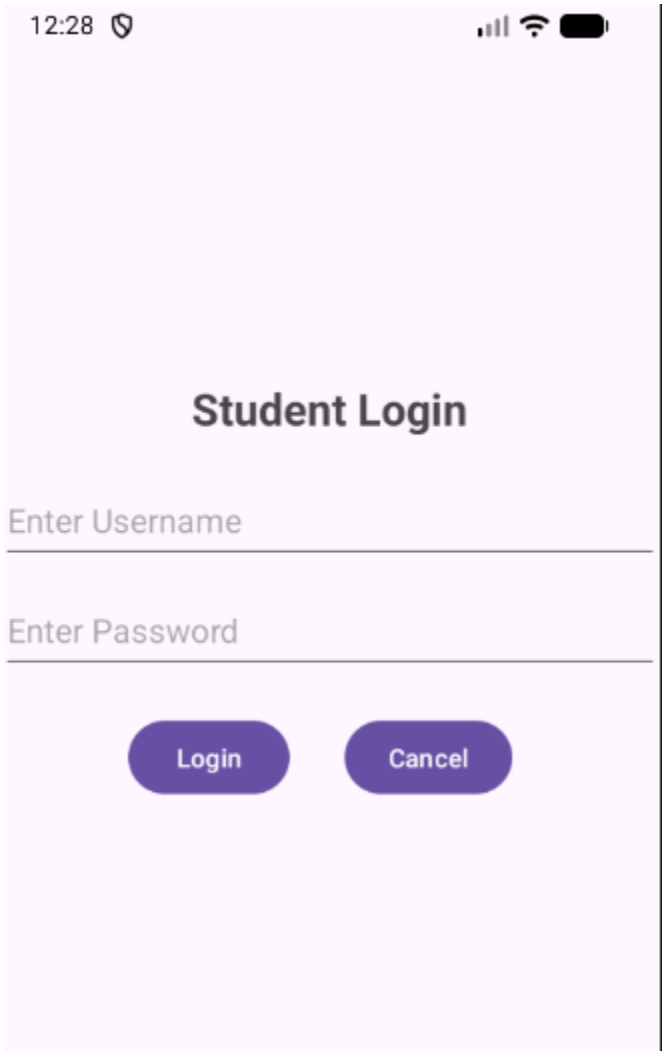
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity"
  android:padding="16dp"
  android:gravity="center">
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

```
  <TextView
     android:id="@+id/txtTitle"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:text="Student Login"
     android:textSize="24sp"
     android:textStyle="bold"
     android:layout_centerHorizontal="true"/>
  <EditText
     android:id="@+id/etUsername"
     android:layout_width="match_parent"
     android:layout_height="50dp"
     android:hint="Enter Username"
     android:layout_below="@id/txtTitle"
     android:layout_marginTop="20dp"
     />
  <EditText
     android:id="@+id/etPassword"
     android:layout_width="match_parent"
     android:layout_height="50dp"
     android:hint="Enter Password"
     android:inputType="textPassword"
     android:layout_below="@id/etUsername"
     android:layout_marginTop="10dp"/>
  <Button
     android:id="@+id/btnLogin"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:text="Login"
     android:layout_below="@id/etPassword"
     android:layout_marginLeft="70dp"
     android:layout_marginTop="20dp"
    />
  <Button
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:text="Cancel"
     android:id="@+id/btncan"
     android:layout_below="@id/etPassword"
     android:layout_marginTop="20dp"
     android:layout_toRightOf="@id/btnLogin"
     android:layout_marginLeft="30dp"
     />
</RelativeLayout>
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No:3-C

Aim: Develop an Android application using TableLayout to design a structured Login/Registration Form.

TableLayout arranges UI components in rows and columns, similar to a table.

- `<TableLayout>` → Creates a table.
- `<TableRow>` → Creates a row.
- UI components like TextView, EditText, Button are placed inside TableRow.

Code:

activity_main.xml

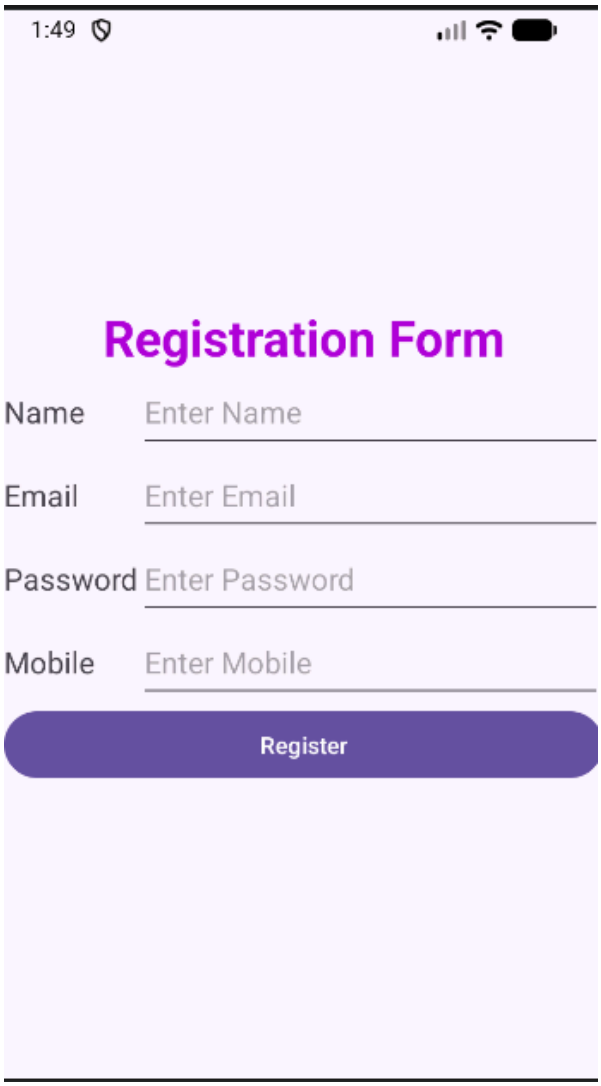
```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity"
  android:padding="20dp"
  android:stretchColumns="1"
  android:gravity="center">
  <TableRow>
  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Registration Form"
    android:textSize="30sp"
    android:layout_span="2"
    android:gravity="center"
    android:textStyle="bold"
    android:textColor="#AF03DA"
    />
  </TableRow>
  <TableRow>
    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Name"
      android:textSize="18sp"/>
    <EditText
      android:id="@+id/etName"
      android:hint="Enter Name"
```

```
        android:layout_width="wrap_content"
        android:layout_height="50dp"/>
    </TableRow>

    <TableRow>
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Email"
            android:textSize="18sp"
            />
        <EditText
            android:id="@+id/etEmail"
            android:hint="Enter Email"
            android:layout_width="wrap_content"
            android:layout_height="50dp"/>
    </TableRow>
    <TableRow>
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Password"
            android:textSize="18sp"/>
        <EditText
            android:id="@+id/etPassword"
            android:hint="Enter Password"
            android:inputType="textPassword"
            android:layout_width="wrap_content"
            android:layout_height="50dp"
            />
    </TableRow>
   <TableRow>
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Mobile"
            android:textSize="18sp"/>
        <EditText
            android:id="@+id/etMobile"
            android:hint="Enter Mobile"
            android:inputType="phone"
            android:layout_width="wrap_content"
            android:layout_height="50dp"/>
    </TableRow>
```

```
  <TableRow>
    <Button
      android:id="@+id/btnRegister"
      android:layout_span="2"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="Register"/>
  </TableRow>
</TableLayout>
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 5-B

Adapter Based Views

Aim: Develop an Android application using GridView with Custom Adapter and image selection handling.

GridView is a ViewGroup that displays items in a two-dimensional scrollable grid.

A **Custom Adapter** is used to display custom layouts (such as images and text) inside GridView cells.

When a user clicks an image, an event is handled using **OnItemClickListener**.

1) **activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<GridView xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/gridView"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity"
  android:numColumns="2"
  android:verticalSpacing="10dp"
  android:horizontalSpacing="10dp"
  android:padding="10dp"/>
```

Step 1

In Android Studio, go to:

```
app
 └── res
     └── layout
```

Step 2

Right-click **layout → New → Layout Resource File**

Step 3

Enter:

File Name : grid_item
Root Element : LinearLayout -> then click on OK

By: Prof. Krutika H. Churi
SDSM College, Palghar

2) **grid_item.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:gravity="center">

  <ImageView
    android:id="@+id/imageView"
     android:layout_width="120dp"
     android:layout_height="120dp"
     android:scaleType="centerCrop"/>

  <TextView
     android:id="@+id/textView"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:textSize="18sp"/>

</LinearLayout>
```

3) CustomAdapter.java (Create class file -CustomAdapter.java–Right-click on your package name. Select:New → Java Class. Enter:CustomAdapter Click **OK**.

```
package com.example.linear_lay;
import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.ImageView;
import android.widget.TextView;
public class CustomAdapter extends BaseAdapter {
  Context context;
  String[] names = {
        "Flower",
        "Nature",
        "Mountain",
        "River"
  };
  int[] images = {
        R.drawable.nature,
        R.drawable.mountain,
        R.drawable.river,
```

```
        R.drawable.flower
    };
    public CustomAdapter(Context context) {
        this.context = context;
    }
    public int getCount() {
        return images.length;
    }
    public Object getItem(int position) {
        return images[position];
    }
    public long getItemId(int position) {
        return position;
    }
    public View getView(int position, View convertView, ViewGroup parent) {
        LayoutInflater inflater = LayoutInflater.from(context);
        View view = inflater.inflate(R.layout.grid_item, parent, false);
        ImageView imageView = view.findViewById(R.id.imageView);
        TextView textView = view.findViewById(R.id.textView);
        imageView.setImageResource(images[position]);
        textView.setText(names[position]);
        return view;
    }
}
```

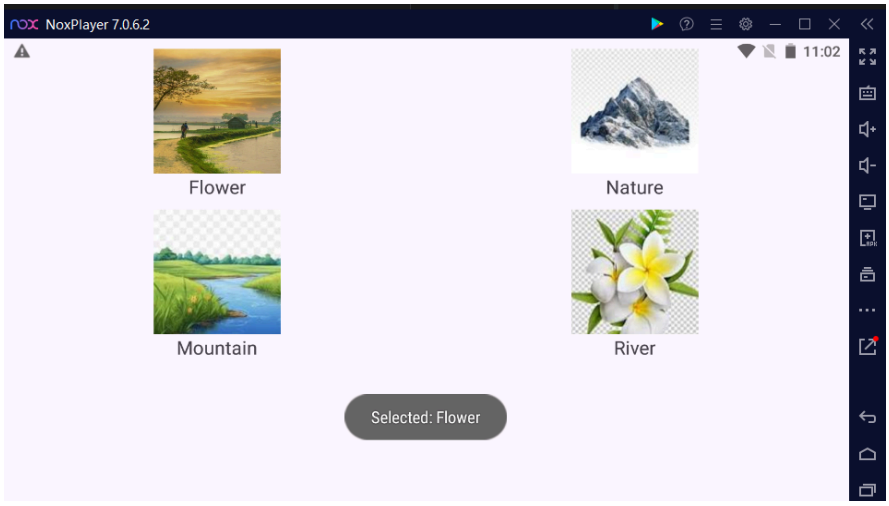
4) MainActivity.java

```
package com.example.linear_lay;
import android.annotation.SuppressLint;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.GridView;
import android.widget.ListView;
import android.widget.Toast;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
    GridView gridView;
    String[] names = {
```

```
            "Flower",
            "Nature",
            "Mountain",
            "River"
    };
    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        gridView = findViewById(R.id.gridView);
        CustomAdapter adapter =
                new CustomAdapter(this);
        gridView.setAdapter(adapter);
        gridView.setOnItemClickListener(
                new AdapterView.OnItemClickListener() {
                    @Override
                    public void onItemClick(
                            AdapterView<?> parent,
                            android.view.View view,
                            int position,
                            long id) {
                        Toast.makeText(MainActivity.this, "Selected: " + names[position],
Toast.LENGTH_SHORT).show();
                    }
                });
    }
}
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No 6-B

Aim: Create an Android application to display an AlertDialog when the user presses the Back button.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:gravity="center">
   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Back Button AlertDialog Demo"
      android:textSize="22sp"
      android:textStyle="bold"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.alertdialogdemo;
import android.os.Bundle;
import androidx.activity.OnBackPressedCallback;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);

      setContentView(R.layout.activity_main);

      // Handle Back Button
      getOnBackPressedDispatcher().addCallback(
            this,
            new OnBackPressedCallback(true) {
               @Override
               public void handleOnBackPressed() {
                  showExitDialog();
               }
```

```
            });
    }
    private void showExitDialog() {

        AlertDialog.Builder builder =
                new AlertDialog.Builder(MainActivity.this);

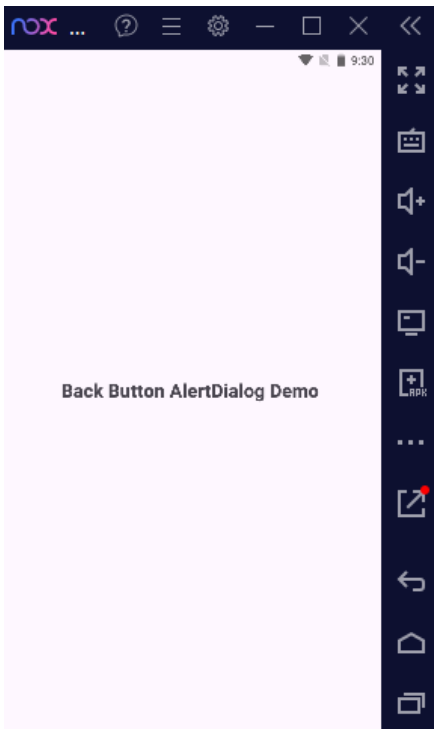
        builder.setTitle("Exit");
        builder.setMessage(
                "Are you sure you want to exit?");

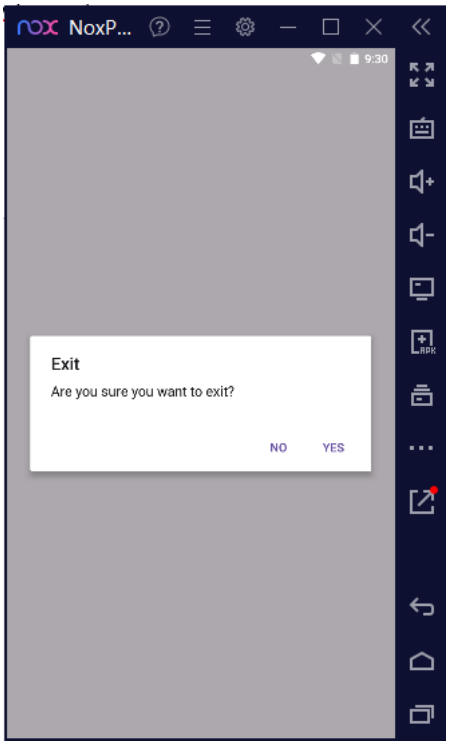
        builder.setPositiveButton("YES",
                (dialog, which) -> {
                    finish();
                });

        builder.setNegativeButton("NO",
                (dialog, which) -> {
                    dialog.dismiss();
                });

        builder.show();
    }
}
```

Output:





By Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 8-B

Aim: Develop an Android application implementing Notifications using Notification Manager.

Activitymain.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:orientation="vertical"
android:layout_height="match_parent"
tools:context=".MainActivity">

<Button
android:id="@+id/click"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_centerInParent="true"
android:text="NOTIFY"/>

</RelativeLayout>
```

ActivityMain.java:

```
package com.example.noti;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;

import android.app.NotificationManager;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

public class MainActivity extends AppCompatActivity {
Button button;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
button=(Button)findViewById(R.id.click);
button.setOnClickListener(new View.OnClickListener() {
@Override
public void onClick(View v) {

NotificationCompat.Builder mbuilder = (NotificationCompat.Builder)
```

```
new NotificationCompat.Builder(getApplicationContext())

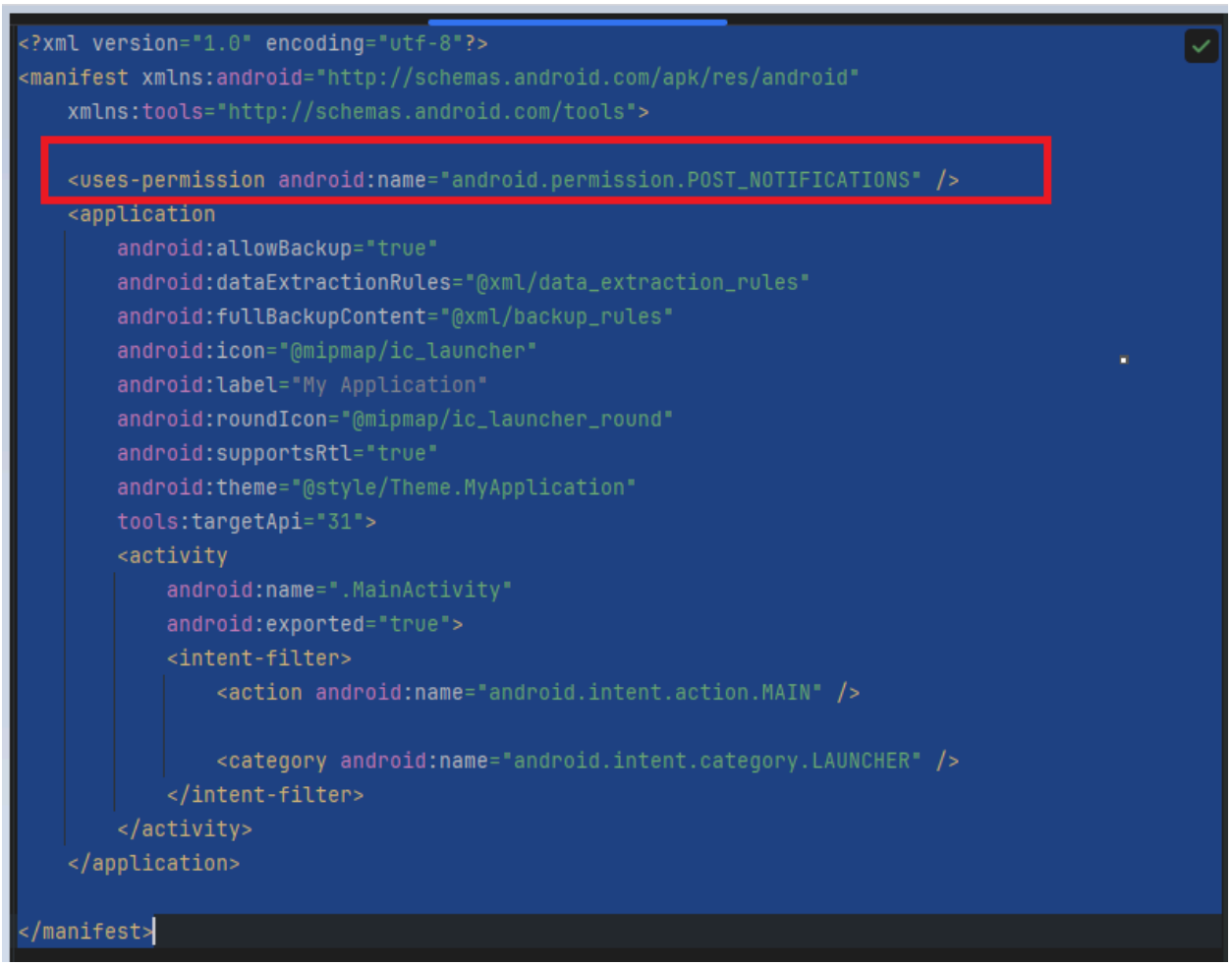
.setSmallIcon(R.drawable.ic_launcher_background)
                .setContentTitle("Notification Example")
                .setContentText("This is a notification message");


NotificationManager notificationManager = (NotificationManager)
                        getSystemService(NOTIFICATION_SERVICE);
notificationManager.notify(0,mbuilder.build());
            }
        });
    }
}
```

AndroidMenifest.xml

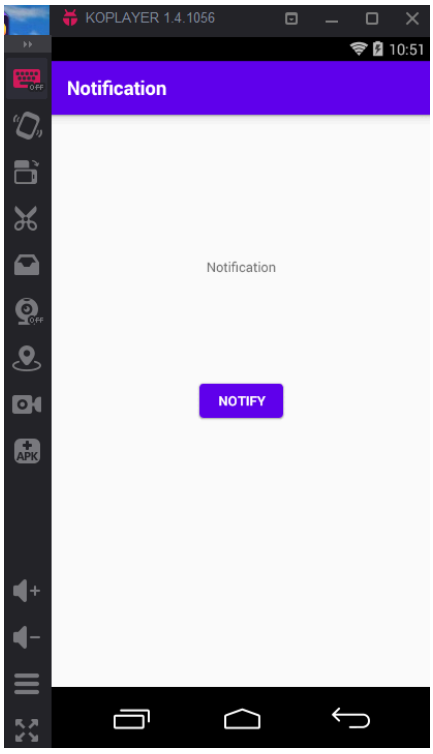
Add this permission following permission to AndroidMenifest.xml file as shown in below image.

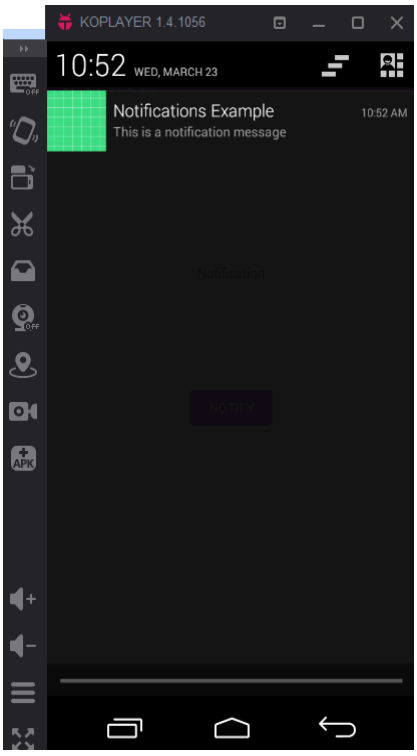
```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"
/>
```



By: Prof. Krutika H. Churi
SDSM College, Palghar

Output:





By: Prof. Krutika H. Churi
SDSM College, Palghar

By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 7-B

Aim: Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

Event Handling

Event Handling is the process of responding to user actions such as:

- Button Click
- Long Press
- Touch Events
- Item Selection

Android provides Listeners to handle these events.

Common Listeners

- `OnClickListener` → **Triggered when a view is clicked.**
- `OnLongClickListener` → **Triggered when a view is pressed and held.**

acivity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity"
   android:orientation="vertical"
   android:gravity="center">

   <Button
       android:id="@+id/btnClick"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:text="Click Me"/>

   <Button
       android:id="@+id/btnLongClick"
       android:layout_width="wrap_content"
       android:layout_height="wrap_content"
       android:text="Long Click Me"
       android:layout_marginTop="20dp"/>
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

```
</LinearLayout>
```

MainActivity.java

```
package com.example.pract1;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.MotionEvent;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
   Button btnClick, btnLongClick;

   @SuppressLint("MissingInflatedId")
   @Override
   protected void onCreate(Bundle savedInstanceState) {
       super.onCreate(savedInstanceState);
       EdgeToEdge.enable(this);
       setContentView(R.layout.activity_main);
       btnClick = findViewById(R.id.btnClick);
       btnLongClick = findViewById(R.id.btnLongClick);

       // OnClick Listener
       btnClick.setOnClickListener(v -> {

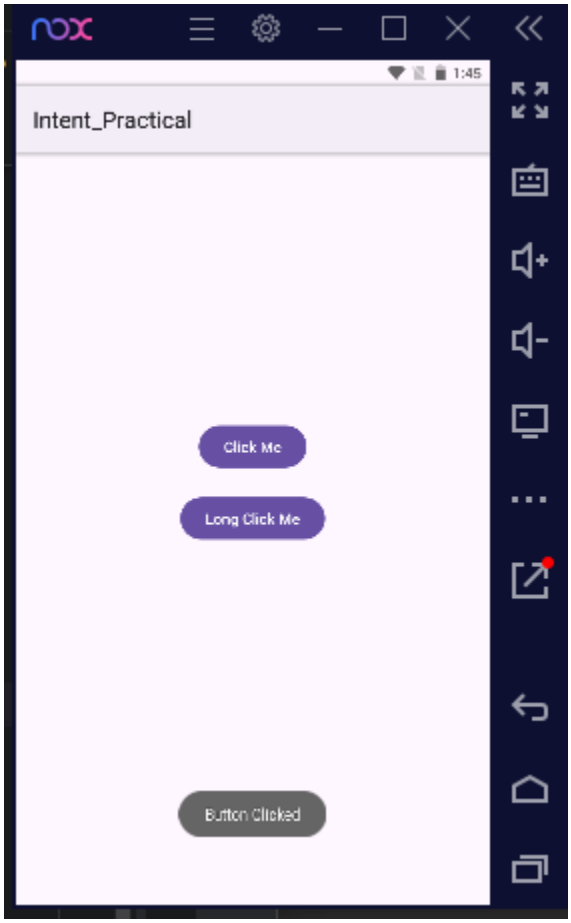
           Toast.makeText(
                   MainActivity.this,
                   "Button Clicked",
                   Toast.LENGTH_SHORT
           ).show();

       });

       // OnLongClick Listener
       btnLongClick.setOnLongClickListener(v -> {
```

```
            Toast.makeText(
                    MainActivity.this,
                    "Button Long Pressed",
                    Toast.LENGTH_SHORT
            ).show();
            return true;
        });
    }
}
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No :8-C

Aim: Create a BroadcastReceiver that listens for the Android Intent ACTION_BATTERY_CHANGED broadcast and shows a Toast displaying the current battery percentage. code in android

This BroadcastReceiver listens for battery change events and displays the current battery percentage using a Toast message.

Step 1: Create BatteryReceiver.java class file

BatteryReceiver.java

```
package com.example.batteryapp;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.BatteryManager;
import android.widget.Toast;

public class BatteryReceiver extends BroadcastReceiver {
   @Override
   public void onReceive(Context context, Intent intent) {
      int level = intent.getIntExtra(BatteryManager.EXTRA_LEVEL, -1);
      int scale = intent.getIntExtra(BatteryManager.EXTRA_SCALE, -1);
      int batteryPercent = (level * 100) / scale;
      Toast.makeText(context,
            "Battery Percentage: " + batteryPercent + "%",
            Toast.LENGTH_LONG).show();
   }
}
```

Step 2: Register Receiver in MainActivity (Dynamic Registration)

MainActivity.java

```
package com.example.batteryapp;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
  BatteryReceiver batteryReceiver;

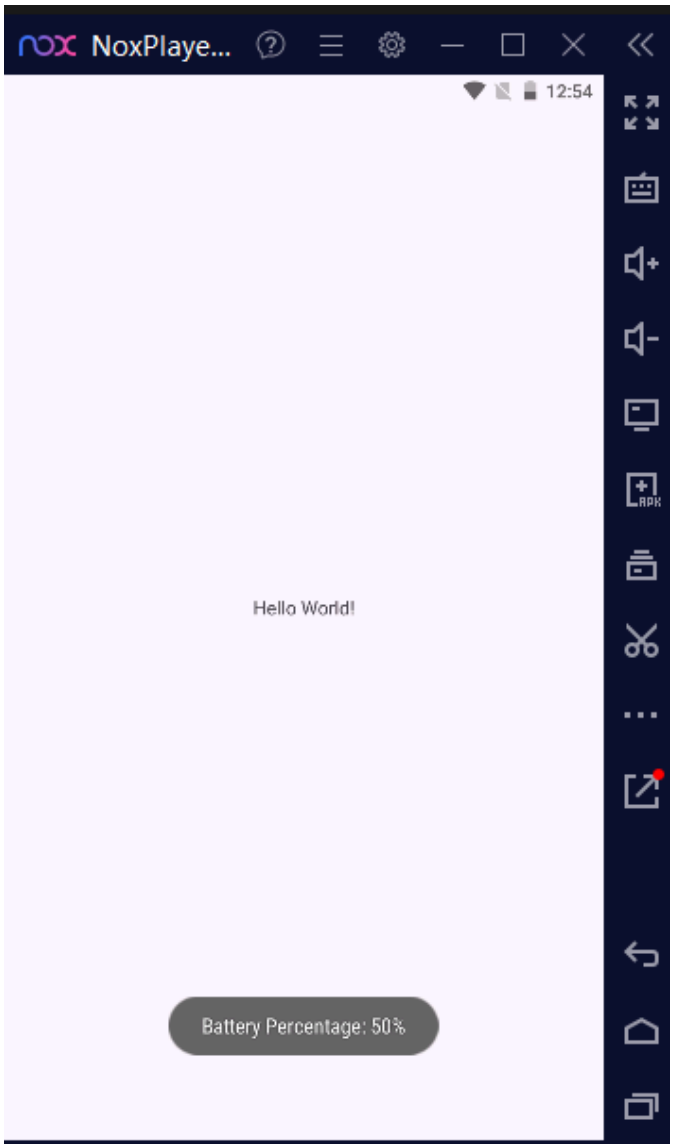
  @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        batteryReceiver = new BatteryReceiver();

        IntentFilter intentFilter = new IntentFilter(Intent.ACTION_BATTERY_CHANGED);
        registerReceiver(batteryReceiver, intentFilter);
    }
}
```

Output:



By Prof. Krutika H. Churi
SDSM College, Palghar

Practical No:7-A

Aim: Develop an Android application implementing Explicit and Implicit Intents.

Intent

An Intent is a messaging object used to communicate between Android components.

There are two types of Intents:

1. Explicit Intent

- An Explicit Intent specifies the exact Activity to be opened.
- Communicates between two activities inside the same application.
- Example: Moving from MainActivity to SecondActivity.

2. Implicit Intent

An Implicit Intent does not specify a particular Activity. Android finds an appropriate application to handle the request.

Communicates between two activities of different applications

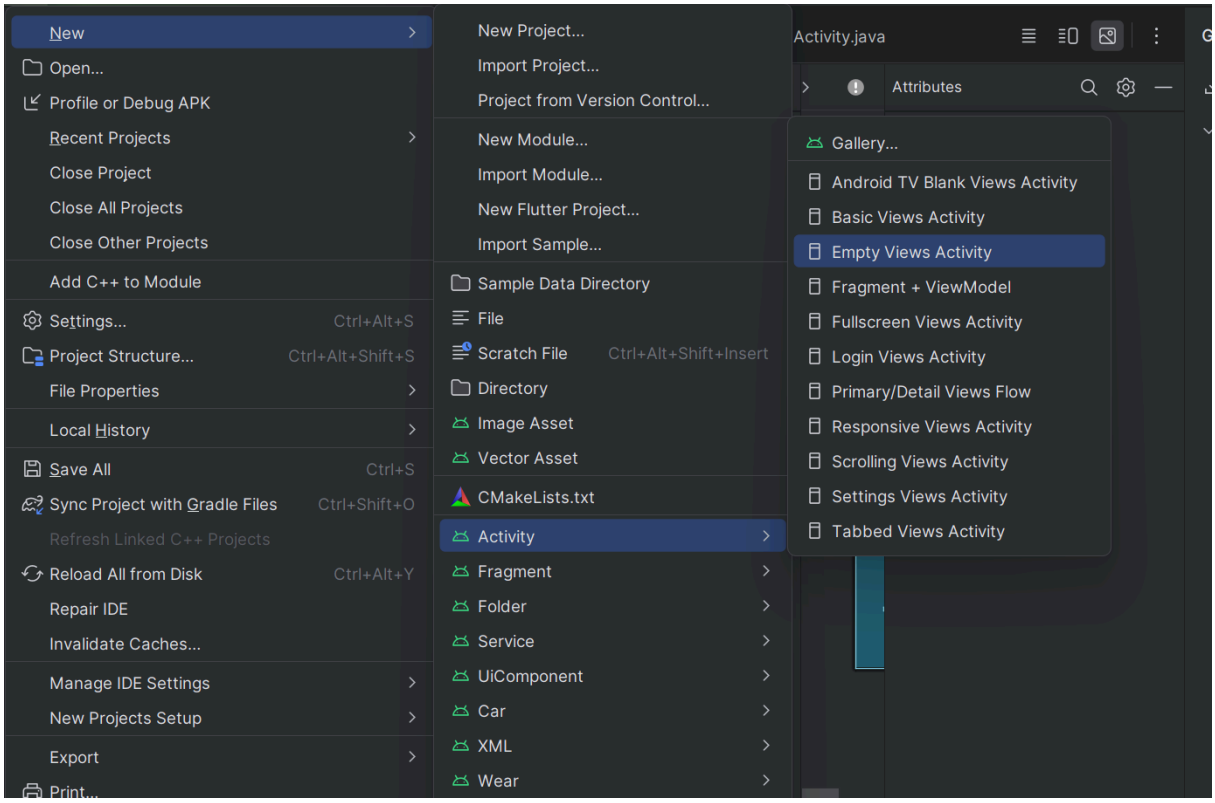
Examples:

- Open a web page
- Make a phone call
- Send an email
- Open Google Maps

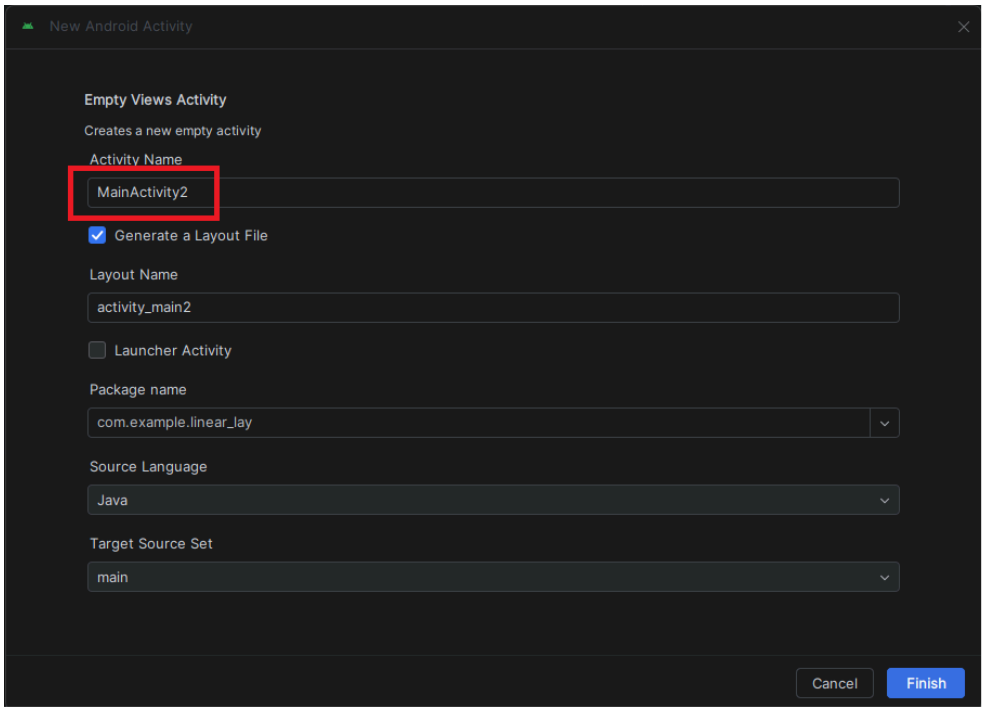
Note: Create Activity_new by using following steps.

```
Right click on app -new -Activity -Empty views activity as shown in
the below screenshot
```

By: Prof. Krutika H. Churi
SDSM College, Palghar



Then set the name for the new activity and click on finish as shown in the below screenshot.



Click on finish.

By: Prof. Krutika H. Churi
SDSM College, Palghar

1) activity _main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity"
  android:gravity="center"
  android:orientation="vertical">

  <Button
    android:id="@+id/btnExplicit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Explicit Intent"/>

  <Button
    android:id="@+id/btnImplicit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Implicit Intent"
    android:layout_marginTop="20dp"/>

</LinearLayout>
```

activity_main2.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity2"
  android:gravity="center">

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Welcome to Second Activity"
    android:textSize="24sp"/>
```

```
</LinearLayout>
```

MainActivity.java

```
package com.example.pract1;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
  Button btnExplicit, btnImplicit;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
     super.onCreate(savedInstanceState);
     EdgeToEdge.enable(this);
     setContentView(R.layout.activity_main);
     btnExplicit = findViewById(R.id.btnExplicit);
     btnImplicit = findViewById(R.id.btnImplicit);

     // Explicit Intent
     btnExplicit.setOnClickListener(v -> {

        Intent intent =
              new Intent(
                    MainActivity.this,
                    MainActivity2.class);

        startActivity(intent);
     });

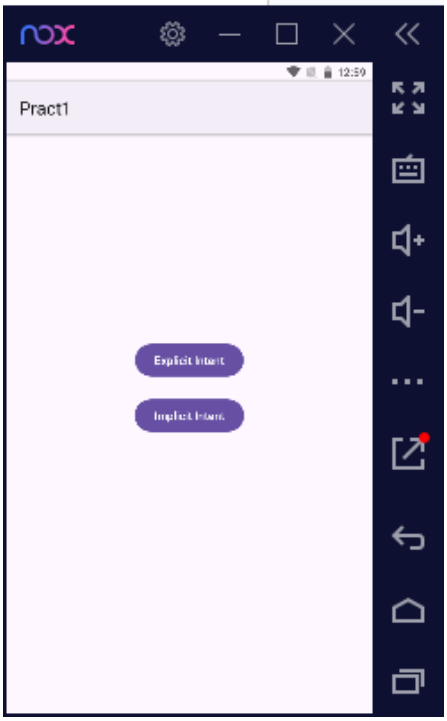
     // Implicit Intent
     btnImplicit.setOnClickListener(v -> {

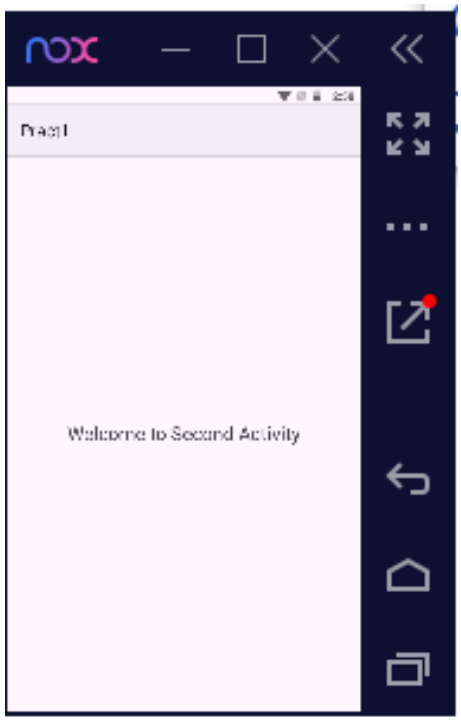
        Intent intent =
              new Intent(
                    Intent.ACTION_VIEW, Uri.parse("https://www.google.com"));
        startActivity(intent);
```

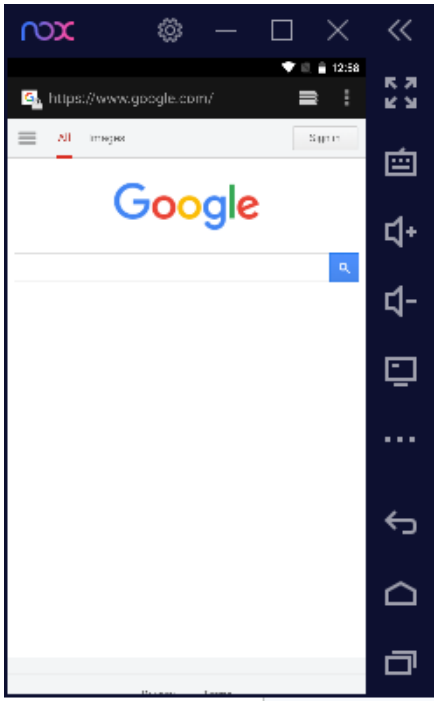
```
        });
    }
}
```

Keep MainActivity2.java as it is

Output:







By: Prof. Krutika H. Churi
SDSM College, Palghar

By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No : 9

Aim: Develop an Android application implementing SQLite database CRUD operations.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity"
  android:orientation="vertical">

  <EditText
    android:id="@+id/id"
    android:hint="ID"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

  <EditText
    android:id="@+id/name"
    android:hint="Name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
  <EditText
    android:id="@+id/lname"
    android:hint="Enter Last Name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

  <Button
    android:id="@+id/insert"
    android:text="INSERT"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

  <Button
    android:id="@+id/view"
    android:text="VIEW"
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"/>

  <Button
    android:id="@+id/update"
    android:text="UPDATE"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

  <Button
    android:id="@+id/delete"
    android:text="DELETE"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

</LinearLayout>
```

DBHelper.java

```
package com.example.database;

import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DBHelper extends SQLiteOpenHelper {

  public DBHelper(Context c) {
    super(c, "StudentDB", null, 1);
  }

  public void onCreate(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE student(" +
        "id INTEGER PRIMARY KEY, name TEXT, lname TEXT)");
  }

  public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("DROP TABLE IF EXISTS student");
    onCreate(db);
  }

  void insert(int id, String name, String lname) {
    getWritableDatabase().execSQL(
        "INSERT INTO student VALUES(" + id + ",'" +
```

```
                name + "','" + lname + "')");
  }

  void update(int id, String name, String lname) {
    getWritableDatabase().execSQL(
        "UPDATE student SET name='" + name +
            "', lname='" + lname + "' WHERE id=" + id);
  }

  void delete(int id) {
    getWritableDatabase().execSQL(
        "DELETE FROM student WHERE id=" + id);
  }

  Cursor view() {
    return getReadableDatabase().rawQuery(
        "SELECT * FROM student", null);
  }
}
```

MainActivity.java

```
package com.example.database;

import android.database.Cursor;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
  EditText id, name,lname;
  Button b1,b2,b3,b4;
  DBHelper db;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
```

```
EdgeToEdge.enable(this);
setContentView(R.layout.activity_main);
id = findViewById(R.id.id);
name = findViewById(R.id.name);
lname = findViewById(R.id.lname);
b1=findViewById(R.id.insert);
b2=findViewById(R.id.update);
b3=findViewById(R.id.delete);
b4=findViewById(R.id.view);
db = new DBHelper(this);

b1.setOnClickListener(v ->
        db.insert(
                Integer.parseInt(id.getText().toString()),
                name.getText().toString(),
                lname.getText().toString()));

b2.setOnClickListener(v ->
        db.update(
                Integer.parseInt(id.getText().toString()),
                name.getText().toString(),
                lname.getText().toString()));

b3.setOnClickListener(v ->
        db.delete(
                Integer.parseInt(id.getText().toString())));

b4.setOnClickListener(v -> {

    Cursor c = db.view();
    String data = "";

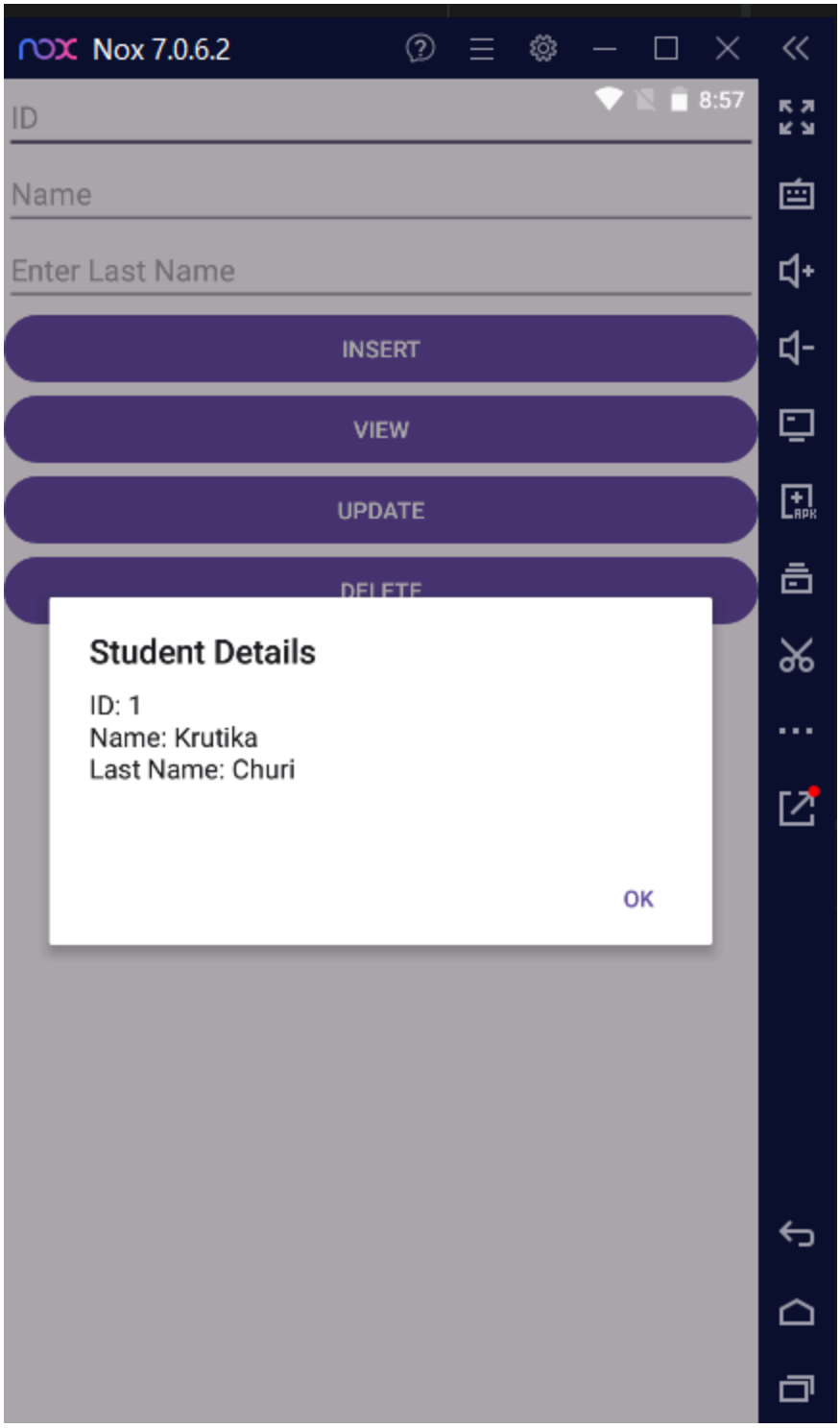
    while (c.moveToNext()) {
        data += "ID: " + c.getInt(0) +
                "\nName: " + c.getString(1) +
                "\nLast Name: " + c.getString(2) +
                "\n\n";
    }

    new AlertDialog.Builder(this)
            .setTitle("Student Details")
            .setMessage(data)
            .setPositiveButton("OK", null)
            .show();
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

```
      });
  }
}
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 6-C

Aim: Develop an Android application implementing contextual menus and popup menus.

Contextual Menu

A Contextual Menu appears when the user performs a long press on a view. It provides actions related to the selected item.

Popup Menu

A Popup Menu displays a list of menu items anchored to a specific view, usually a button.

A) activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity"
  android:orientation="vertical"
  android:gravity="center">

  <TextView
    android:id="@+id/txtContext"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Long Press Me"
    android:textSize="22sp"
    android:padding="20dp"/>

  <Button
    android:id="@+id/btnPopup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Show Popup Menu"
    android:layout_marginTop="20dp"/>

</LinearLayout>
```

Note: 1) To create **Context_menu.xml and popup_menu.xml**
file Right click on res–Click on new-android resource file==set file name=**Context_menu** and resource

type =menu repeat same to create **popup_menu.xml**

1. Context_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

  <item
     android:id="@+id/edit"
     android:title="Edit"/>

  <item
     android:id="@+id/delete"
     android:title="Delete"/>

</menu>
```

2. popup_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <item
     android:id="@+id/share"
     android:title="Share"/>

  <item
     android:id="@+id/download"
     android:title="Download"/>

</menu>
```

B) MainActivity.java

```
package com.example.menu_pract;

import android.os.Bundle;
import android.view.ContextMenu;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;
```

```
import androidx.appcompat.widget.Toolbar;

import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
  TextView txtContext;
  Button btnPopup;
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    txtContext = findViewById(R.id.txtContext);
    btnPopup = findViewById(R.id.btnPopup);
    registerForContextMenu(txtContext);
    btnPopup.setOnClickListener(v -> {
            PopupMenu popupMenu = new PopupMenu(MainActivity.this,btnPopup);
            popupMenu.getMenuInflater()
                    .inflate(R.menu.popup_menu,
                            popupMenu.getMenu());
      popupMenu.setOnMenuItemClickListener(item -> {

        if(item.getItemId()==R.id.share)
        {
          Toast.makeText(
                  MainActivity.this,
                  "Share Selected",
                  Toast.LENGTH_SHORT).show();
        }
        else if(item.getItemId()==R.id.download)
        {
          Toast.makeText(
                  MainActivity.this,
                  "Download Selected",
                  Toast.LENGTH_SHORT).show();
        }

        return true;
      });
```

```
            popupMenu.show();
        });
    }

    // Context Menu Creation
    @Override
    public void onCreateContextMenu(
            ContextMenu menu,
            View v,
            ContextMenu.ContextMenuInfo menuInfo) {

        super.onCreateContextMenu(menu, v, menuInfo);

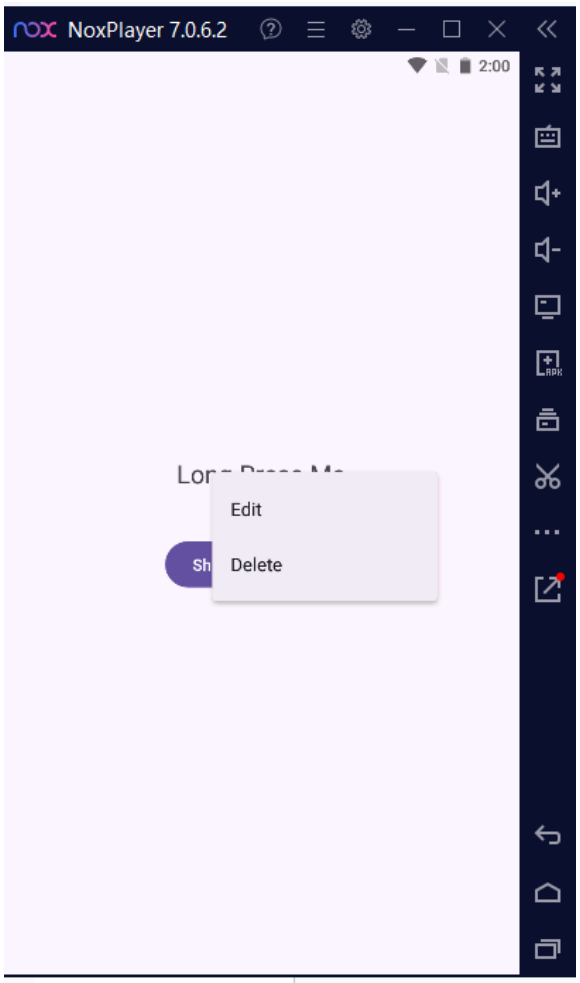
        MenuInflater inflater = getMenuInflater();
        inflater.inflate(R.menu.context_menu, menu);
    }

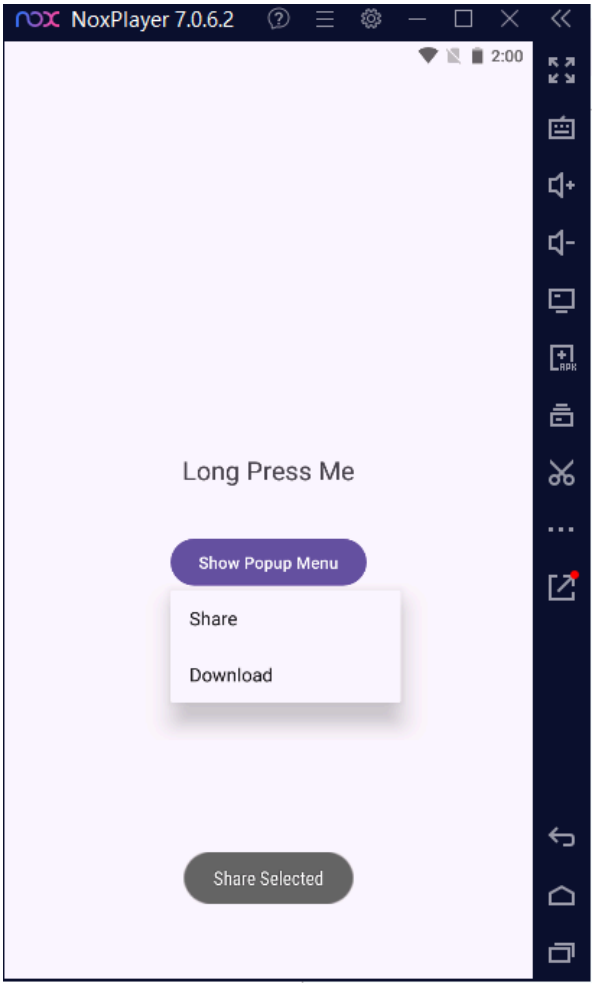
    // Context Menu Item Click
    @Override
    public boolean onContextItemSelected(MenuItem item) {

        if(item.getItemId()==R.id.edit)
        {
            Toast.makeText(this,
                    "Edit Selected",
                    Toast.LENGTH_SHORT).show();
        }
        else if(item.getItemId()==R.id.delete)
        {
            Toast.makeText(this,
                    "Delete Selected",
                    Toast.LENGTH_SHORT).show();
        }

        return true;
    }
}
```

Output:





By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 8-B

Aim: Create an android application which automatically notify the user when Airplane mode is turned on or off using a broadcast receiver.

Activitymain.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

<ImageView
android:id="@+id/imageView"
android:layout_width="40dp"
android:layout_height="40dp"
android:layout_margin="8dp"
android:layout_marginTop="16dp"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent"
app:srcCompat="@drawable/airplane" />

<TextView
android:id="@+id/textView"
android:layout_width="300dp"
android:layout_height="36dp"
android:layout_marginEnd="8dp"
android:layout_marginStart="8dp"
android:gravity="center_vertical"
android:text="Flight Mode"

android:textSize="24dp"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toEndOf="@+id/imageView"
app:layout_constraintTop_toTopOf="@+id/imageView" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

AndroidMenifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.example.broadcastreceiver">

<application
android:allowBackup="true"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
```

By Prof. Krutika H. Churi
SDSM College, Palghar

```
android:theme="@style/Theme.BroadcastReceiver">
<activity android:name=".MainActivity">
<intent-filter>
<action android:name="android.intent.action.MAIN" />

<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
<receiver
            android:name=".MyReceiver"
            android:enabled="true"
            android:exported="true">
<intent-filter>
<action android:name="android.intent.action.AIRPLANE_MODE"/>
</intent-filter>
</receiver>
</application>
</manifest>
```

MyReceiver.java:

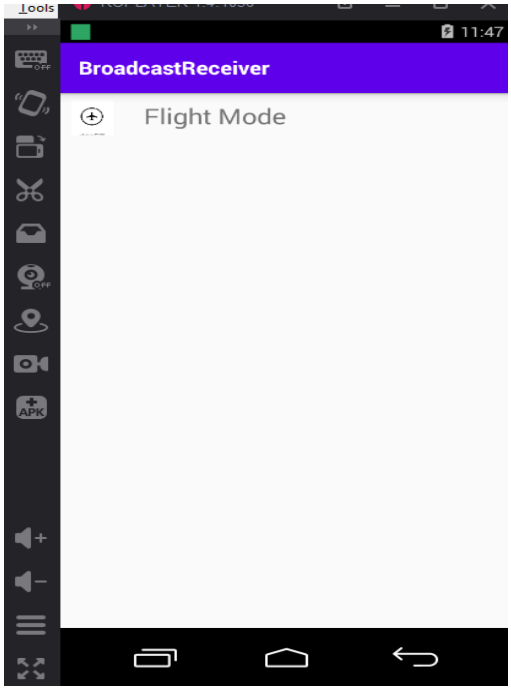
```
package com.example.broadcastreceiver;

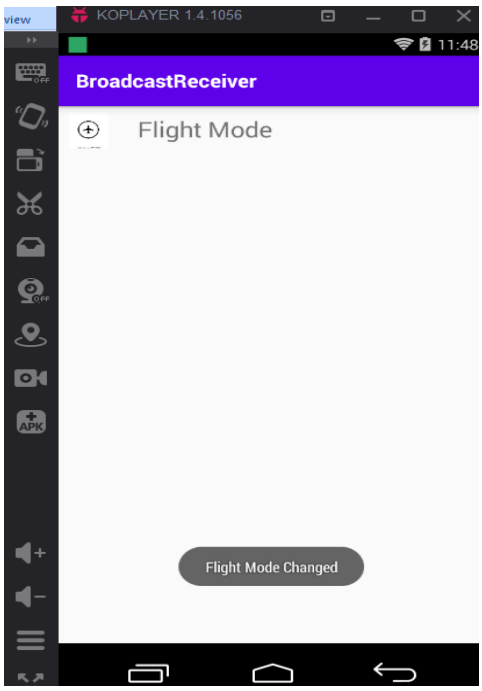
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

public class MyReceiver extends BroadcastReceiver {
public void onReceive(Context context, Intent intent) {
Toast.makeText(context, "Flight Mode Changed",
Toast.LENGTH_LONG).show();
    }

}
```

***Keep MainActivity.Java file as it is





By Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 7-C

Aim: Develop an Android application using Adapters to populate Spinner or ListView dynamically.

Spinner

A Spinner is a dropdown list that allows users to select one item from multiple options.

Adapter

An Adapter acts as a bridge between the data source and the UI component.

ArrayAdapter

An ArrayAdapter is used to connect an array of data to a Spinner or ListView.

activitymain.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:orientation="vertical"
    android:gravity="center">

    <Spinner
        android:id="@+id/spinnerSubjects"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

MainActivity.java

```
package com.example.pract1;

import android.annotation.SuppressLint;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.MotionEvent;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.Spinner;
import android.widget.TextView;
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

```
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

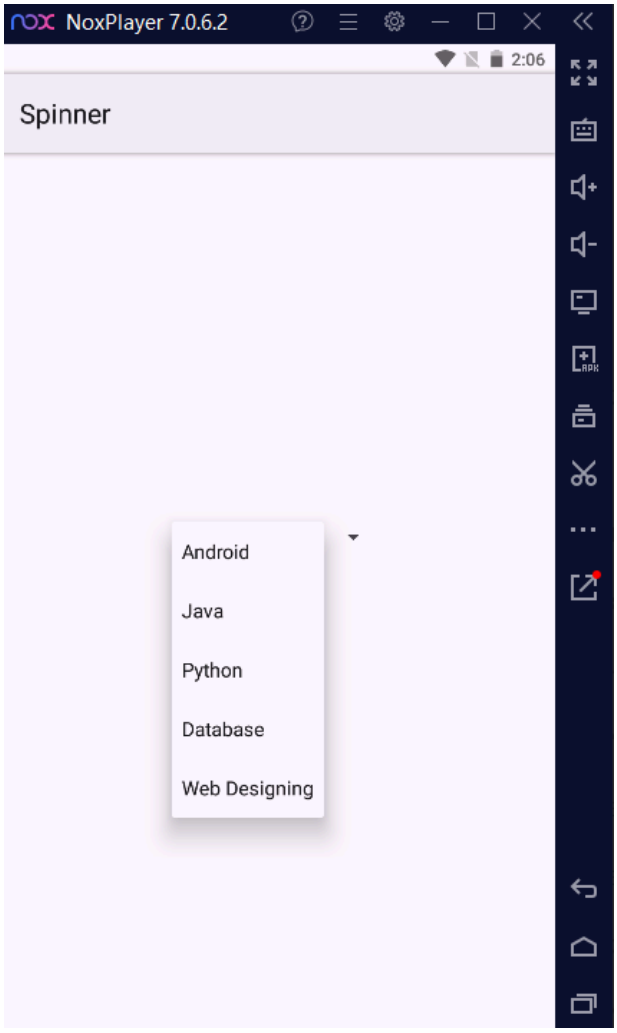
public class MainActivity extends AppCompatActivity {
  Spinner spinnerSubjects;
  String[] subjects = {
        "Android",
        "Java",
        "Python",
        "Database",
        "Web Designing"
  };
  @SuppressLint("MissingInflatedId")
  @Override
  protected void onCreate(Bundle savedInstanceState) {
     super.onCreate(savedInstanceState);
     EdgeToEdge.enable(this);
     setContentView(R.layout.activity_main);
spinnerSubjects =
           findViewById(R.id.spinnerSubjects);
     ArrayAdapter<String> adapter =
           new ArrayAdapter<>(
                this,
                android.R.layout.simple_spinner_item,
                subjects);
     adapter.setDropDownViewResource(
           android.R.layout.simple_spinner_dropdown_item);
     spinnerSubjects.setAdapter(adapter);
        spinnerSubjects.setOnItemSelectedListener(
                  new AdapterView.OnItemSelectedListener() {
                      @Override
                      public void onItemSelected(AdapterView<?> parent,
View view, int position, long id) {
                   Toast.makeText(
                                 MainActivity.this,
                                 "Selected: " + subjects[position],
                                 Toast.LENGTH_SHORT
                         ).show();
                      }
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

```
        @Override
        public void onNothingSelected(AdapterView<?> parent) {

                        }
    });
}
}
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 5-A

Aim: To develop an Android application using ListView with ArrayAdapter and implement item click handling.

ListView

A ListView is a view component used to display a list of items in a scrollable format.

ArrayAdapter

An ArrayAdapter acts as a bridge between the data source (array) and the ListView.

Item Click Handling

When a user clicks an item in the ListView, an action can be performed using the setOnItemClickListener() method.

---

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical">
  <ListView
     android:id="@+id/listView"
     android:layout_width="match_parent"
     android:layout_height="match_parent"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.adapter_pract;

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
```

```
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
  ListView listView;

  String[] subjects = {
        "Android",
        "Java",
        "Python",
        "Database",
        "Web Designing",
        "Data Structures",
        "Networking"
  };
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    listView = findViewById(R.id.listView);

    ArrayAdapter<String> adapter =
          new ArrayAdapter<>(
                this,
                android.R.layout.simple_list_item_1,
                subjects);

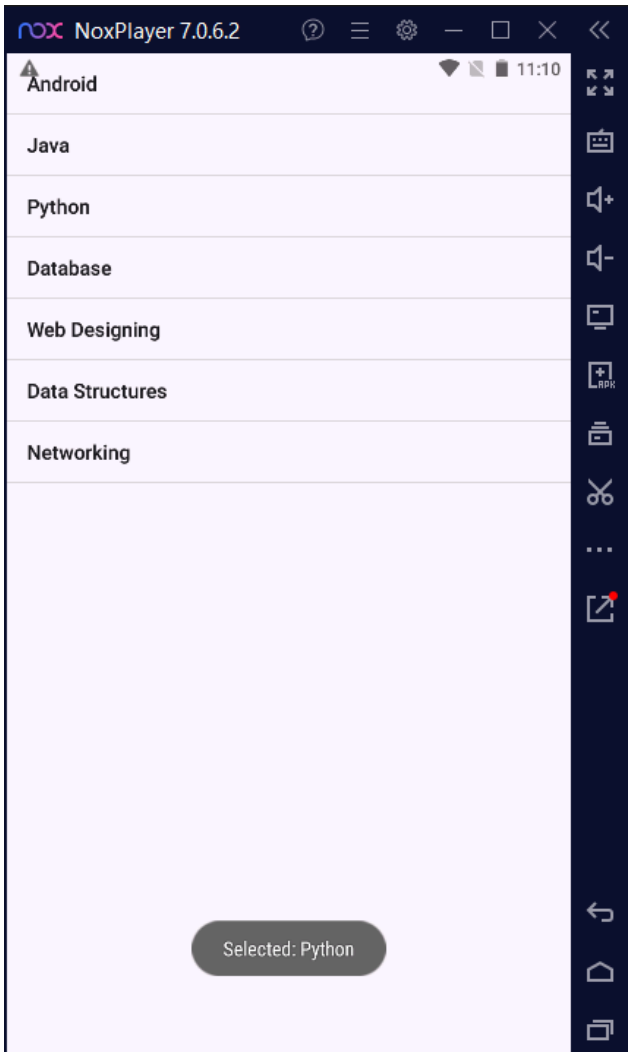
    listView.setAdapter(adapter);

    listView.setOnItemClickListener(
          new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> parent,
                                    View view,
                                    int position,
                                    long id) {

              Toast.makeText(
                    MainActivity.this,
                    "Selected: " + subjects[position],
                    Toast.LENGTH_SHORT
              ).show();
```

```
            }
        });
    }
}
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 4-A

Advanced Layout and View Handling

Aim: Develop an Android application using ScrollView to display long scrollable content.

ScrollView is a layout container that allows users to scroll vertically when the content exceeds the screen size.

Features:

- Enables vertical scrolling.
- Contains only one direct child view.
- Commonly used with LinearLayout.
- Useful for forms, articles, and long text content.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity">

  <LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    >

    <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="ANDROID PROGRAMMING"
      android:textSize="24sp"
      android:textStyle="bold"
      android:paddingTop="20dp"/>

    <TextView
      android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
    android:text="Unit 1: Introduction to Android"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 2: Activities and Intents"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 3: Layouts and Views"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 4: Event Handling"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 5: Fragments"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 6: Menus and Dialogs"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 7: SQLite Database"
    android:paddingTop="20dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Unit 8: Networking"
```

```
        android:paddingTop="20dp"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Unit 9: Firebase"
        android:paddingTop="20dp"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Unit 10: Project Development"
        android:paddingTop="20dp"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This content is intentionally long to demonstrate vertical scrolling in Android using ScrollView. Students can continue scrolling to view additional information on the screen."
        android:paddingTop="20dp"/>

  </LinearLayout>

</ScrollView>
```

Output:



Module 2
Practical No 6-A
Menus, Dialogs and UI Interaction

Aim: Develop an Android application implementing AppBar (Toolbar) and Options Menu.

Menu

- In Android, the menu is a vital UI element that is used to offer certain common functionality throughout the application.
- With the help of the menu, users can experience a smooth and consistent experience throughout the application.
- Menu needs to be defined in a separate XML file, which we should then use in our programme in accordance with our needs.

AppBar (Toolbar)

The **Toolbar** is a modern and flexible version of the ActionBar. It appears at the top of the screen and displays the application title, navigation buttons, and menu items.

Options Menu

The **Options Menu** provides a collection of actions or commands for the user. It appears when the user clicks the menu icon on the Toolbar.

Note: 1) To create menu.xml file Right click on res–Click on new resource file==set file name=mymenu and resource type =menu

1) activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity">

<androidx.appcompat.widget.Toolbar
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:id="@+id/toolbar"
  android:background="#3F51B5"
  app:layout_constraintEnd_toEndOf="parent"
```

```
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Menu Practical"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

2) mymenu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item android:id="@+id/search"
        android:title="Search"/>
    <item android:id="@+id/share"
        android:title="Share"/>
    <item android:id="@+id/app_bar_delete"
        android:title="Delete"/>
    <item android:id="@+id/app_bar_save"
        android:title="Save"/>
    <item android:id="@+id/app_bar_exit"
        android:title="Exit"
        />

</menu>
```

3) MainActivity.java

```
package com.example.menu_pract;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.appcompat.widget.Toolbar;
import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
```

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

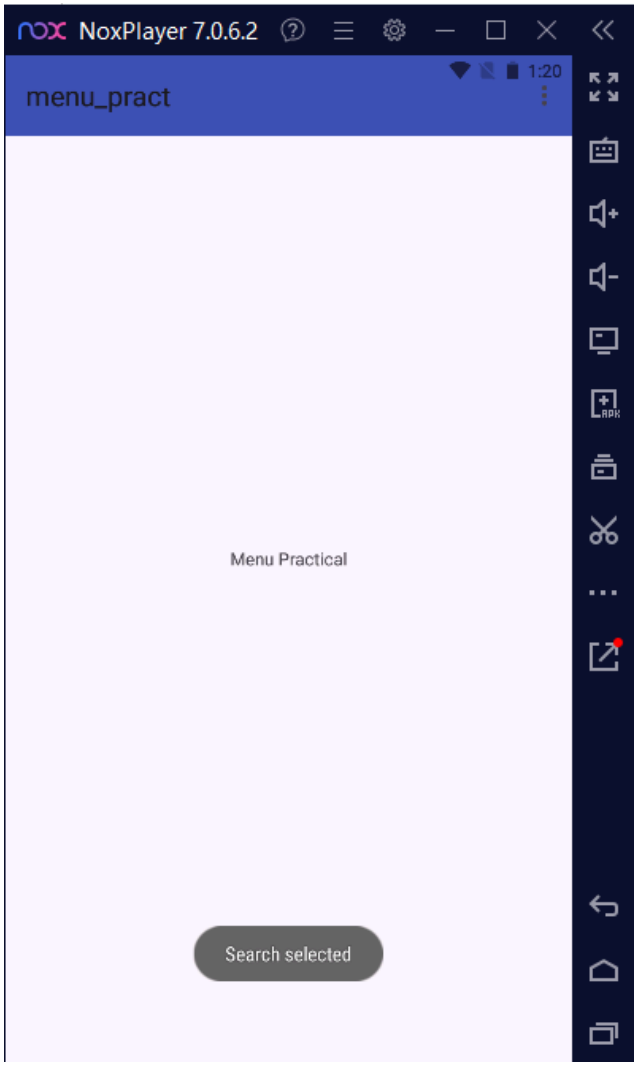
public class MainActivity extends AppCompatActivity {
  Toolbar t1;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
     super.onCreate(savedInstanceState);
     EdgeToEdge.enable(this);
     setContentView(R.layout.activity_main);
     t1 = findViewById(R.id.toolbar);
     setSupportActionBar(t1);
  }
  @Override
  public boolean onCreateOptionsMenu(Menu menu) {
     MenuInflater menuInflater = new MenuInflater(this);
     menuInflater.inflate(R.menu.mymenu,menu);
     return true;
  }

  @Override
  public boolean onOptionsItemSelected(@NonNull MenuItem item) {
     int id = item.getItemId();
     if (id == R.id.search) {
        Toast.makeText(this, "Search selected", Toast.LENGTH_SHORT).show();
        return true;
     }
     else if (id == R.id.app_bar_save) {
        Toast.makeText(this, "Save selected", Toast.LENGTH_SHORT).show();
        return true;
     }
     else if (id == R.id.share) {
        Toast.makeText(this, "Share selected", Toast.LENGTH_SHORT).show();
        return true;
     }
     else if (id == R.id.app_bar_exit) {
        Toast.makeText(this, "Exit selected", Toast.LENGTH_SHORT).show();
        return true;
     }
     else if (id == R.id.app_bar_delete) {
        Toast.makeText(this, "Delete selected", Toast.LENGTH_SHORT).show();
        return true;
```

```
        }

        return super.onOptionsItemSelected(item);
    }
}
```

Output:



Note: import this class if getting error in **setSupportActionBar(t1);**

```
import androidx.appcompat.widget.Toolbar;
```
And remove existing toolbar class

By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No : 4-B

Aim: Develop an Android application using AbsoluteLayout demonstrating fixed positioning.

AbsoluteLayout is a layout that allows you to place UI components at fixed X and Y coordinates on the screen.

In AbsoluteLayout, the position of a view is specified using:

Property	Description
`android:layout_x`	Specifies horizontal position
`android:layout_y`	Specifies vertical position

Code:

1) **activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<AbsoluteLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity">
  <ImageView
     android:id="@+id/imgProfile"
     android:layout_width="120dp"
     android:layout_height="120dp"
    android:layout_x="160dp"
     android:layout_y="40dp"
     android:src="@drawable/user"/>

  <!-- Name Text -->
  <TextView
     android:id="@+id/txtName"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:layout_x="180dp"
```

```
    android:layout_y="180dp"
    android:text="John Doe"
    android:textSize="20sp"
    android:textColor="#D11313"
    android:textStyle="bold"/>

  <!-- Button -->
  <Button
    android:id="@+id/btnViewProfile"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="View Profile"
    android:layout_x="160dp"
    android:layout_y="220dp"/>
</AbsoluteLayout>
```

2) MainActivity.java

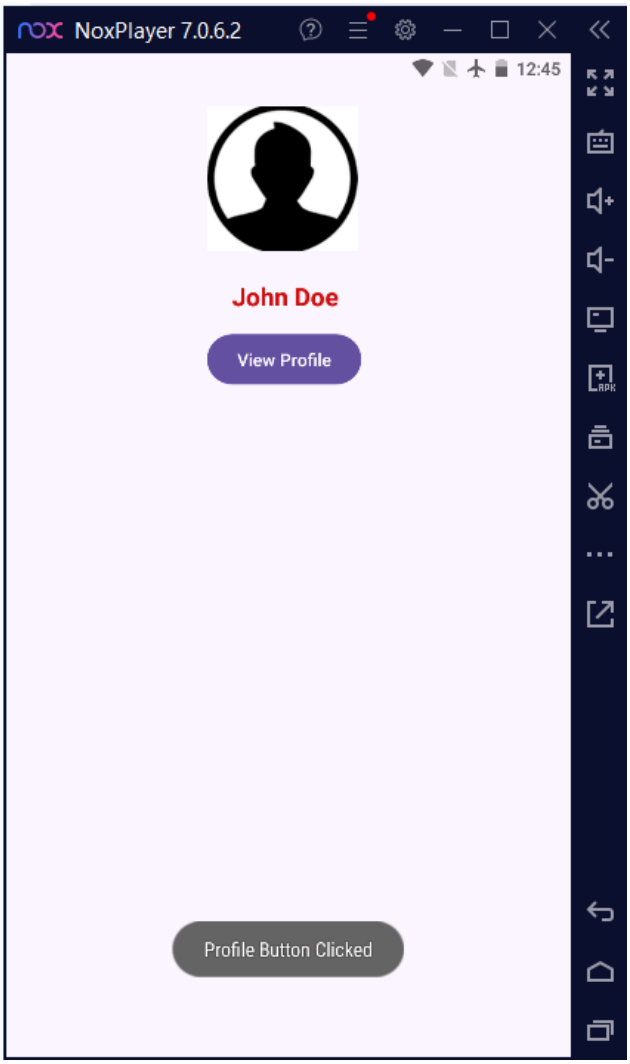
```
package com.example.scroll_view_pract;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {
Button b1;
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    b1=findViewById(R.id.btnViewProfile);
    b1.setOnClickListener(new View.OnClickListener() {
      @Override
      public void onClick(View v) {
        Toast.makeText(MainActivity.this, "Profile Button Clicked",
Toast.LENGTH_SHORT).show();

      }
```

```
        });
    }
}
```

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical N0 4-C

Aim:.Develop an Android application using FrameLayout to overlay UI elements.

FrameLayout is a ViewGroup that displays its child views stacked on top of each other.

- The first view is placed at the bottom.
- The next views are placed above it.
- Commonly used for:
    - Fragments
    - Image overlays
    - Labels on images
    - Custom UI designs

Code:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/main"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity">

  <ImageView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:src="@drawable/nature"
    android:scaleType="centerInside"/>

  <!-- Text Over Image -->

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:text="Beatiful Nature"
    android:textColor="@color/design_default_color_primary"
    android:textSize="24sp"
    android:textStyle="bold" />
</FrameLayout>
```

By: Prof. Krutika H. Churi
SDSM College, Palghar

Output:



By: Prof. Krutika H. Churi
SDSM College, Palghar

Practical No: 2-B

Aim: To create and integrate a Fragment in an Android application and display its user interface inside an Activity.

Code:

Note: 1) Keep MainActivity.java and fragment code for java as it is.

2) Create two fragments- Right click on App – New-Fragment-Fragment(Blank)-Set Name First_Fragment-Click on finish follow the same process to create Second_Fragment.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity"
   android:orientation="vertical">

   <fragment
      android:id="@+id/myFragment"
      android:name="com.example.twoa.MyFragment"
      android:layout_width="match_parent"
      android:layout_height="0dp"
      android:layout_weight="1"
      />
   <fragment
      android:layout_width="match_parent"
      android:layout_height="0dp"
      android:layout_weight="1"
      android:name="com.example.twoa.BlankFragment"
      />
</LinearLayout>
```

fragment_first.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MyFragment"
    android:background="#E33E76">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="First Fragment"
        android:gravity="center"
        android:textSize="25sp"
        android:textStyle="bold"
        android:textColor="#40044A"/>
</FrameLayout>
```

fragment_second.xml

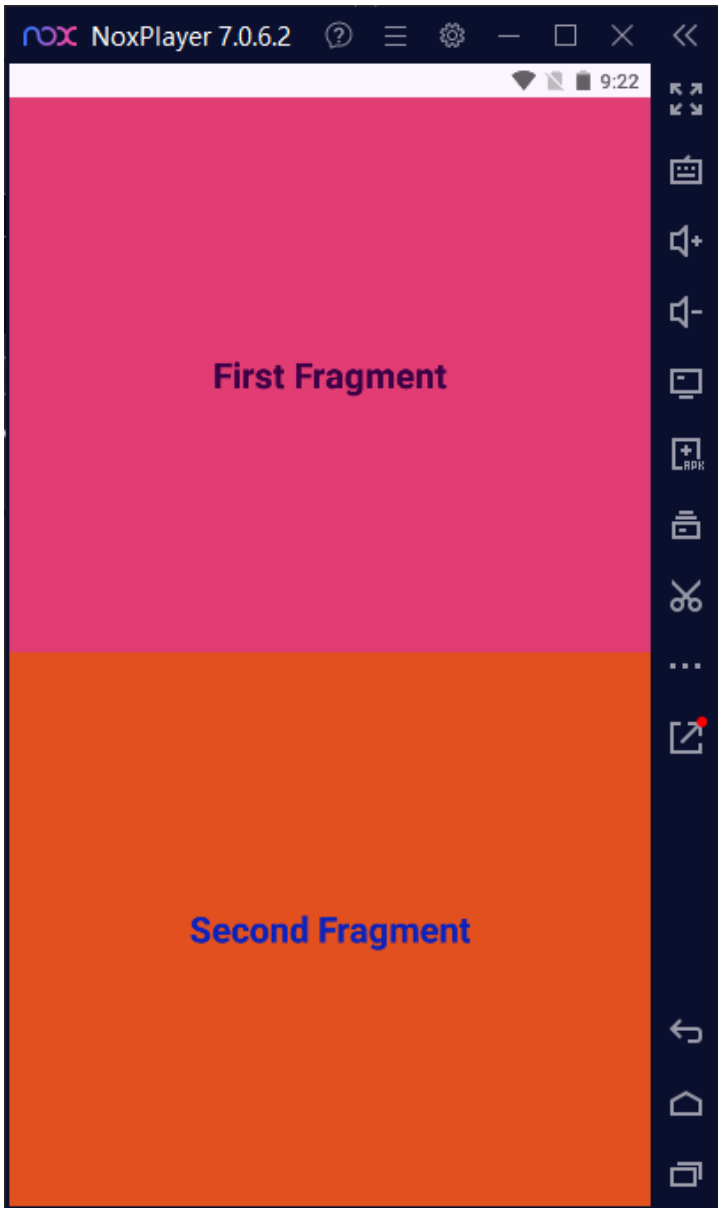
```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".BlankFragment"
    android:background="#E55222">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Second Fragment"
        android:gravity="center"
        android:textSize="25sp"
        android:textStyle="bold"
```

```
        android:textColor="#0B26C0"/>

</FrameLayout>
```

Output:



Practical No. 1-C

Aim: Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello World!"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
app:layout_constraintRight_toRightOf="parent"
app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.java

```
package com.example.actlifecycle;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.Toast;
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
Toast.makeText(this, "Oncreate", Toast.LENGTH_SHORT).show();
}
@Override
protected void onStart() {
super.onStart();
Toast.makeText(this, "OnStart", Toast.LENGTH_SHORT).show();
}

@Override
protected void onRestart() {
super.onRestart();
Toast.makeText(this, "OnRestart", Toast.LENGTH_SHORT).show();
}

@Override
protected void onResume() {
```

```
super.onResume();
Toast.makeText(this, "OnResume", Toast.LENGTH_SHORT).show();
}

@Override
protected void onPause() {
super.onPause();
Toast.makeText(this, "OnPause", Toast.LENGTH_SHORT).show();
}
@Override
protected void onStop() {
super.onStop();
Toast.makeText(this, "OnStop", Toast.LENGTH_SHORT).show();
}
@Override
protected void onDestroy() {
super.onDestroy();
Toast.makeText(this, "OnDestroy", Toast.LENGTH_SHORT).show();
}
}
```

Output:



Practical No:1 -B

Aim: Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.

Android provides resource files to separate UI data from code, making applications easier to manage and maintain.

Resource Files

- strings.xml → Stores text values.
- colors.xml → Stores color values.
- dimens.xml → Stores dimensions such as text size and margins.
- themes.xml → Defines the application's theme.
- drawable → Stores images, shapes, and backgrounds.

Define following colors in Color.xml code:-

Go to res-values-color.xml and open the color.xml file and add the following colors to it.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="Red">#C30A0A</color>
    <color name="Green">#23BF29</color>
    <color name="Purple">#CB4FE0</color>
</resources>
```

Activicty_main.xml code:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"

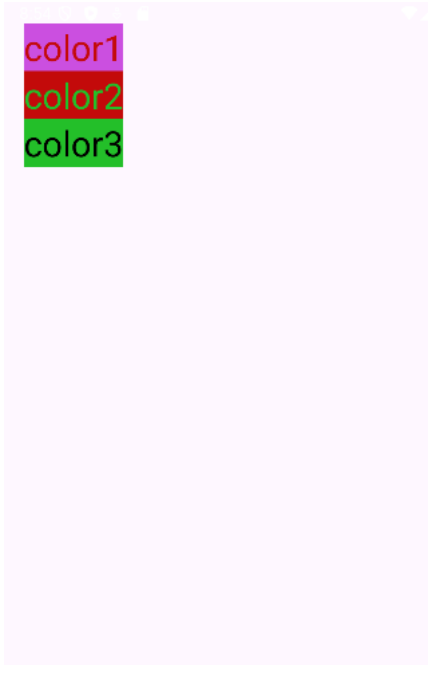
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="color1"
         android:textColor="@color/Red"
        android:textSize="30sp"
        android:background="@color/Purple"/>
```

```
    <TextView
        android:id="@+id/textView4"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@color/Red"
        android:text="color2"
        android:textColor="@color/Green"
        android:textSize="30sp"/>

    <TextView
        android:id="@+id/textView3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="@color/black"
        android:background="@color/Green"
        android:text="color3"
        android:textSize="30sp"/>
</LinearLayout>
```

Output:-



2) Theme

Go to res-values-themes and open thems.xml file

themes.xml code:

activity_main.xml | themes.xml | colors.xml | MainActivity.java

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Audio" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.Audio" parent="Base.Theme.Audio" />
</resources>
```

activity_main.xml | themes.xml | colors.xml | MainActivity.java

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Audio" parent="Theme.Material3.Dark">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.Audio" parent="Base.Theme.Audio" />
</resources>
```

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Audio" parent="Theme.Material3.DayNight">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.Audio" parent="Base.Theme.Audio" />
</resources>
    </resources>
```

```
activity_main.xml | MainActivity.java | strings.xml | themes.xml
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Audio" parent="Theme.Material3.Light">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.Audio" parent="Base.Theme.Audio" />
</resources>
```

3) String

Go to res-values-string and open string.xml file

Write following code to string.xml file

String.xml code:-

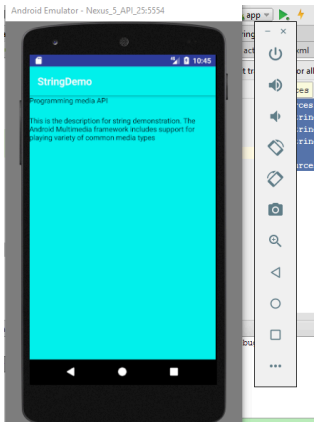
```
<resources>
<string name="app_name">StringDemo</string>
<string name="ParaHead">Programming media API</string>
<string name="Description">This is the description for string demonstration. The Android Multimedia framework includes support for playing variety of common media types</string>
</resources>
```

Activity_main.xml code:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="com.example.student.pract1.MainActivity"
    android:layout_weight="3">
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/ParaHead"/>
<TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Description" android:layout_marginTop="20dp" />
</LinearLayout>
```

Output:-



4) Drawables and images.

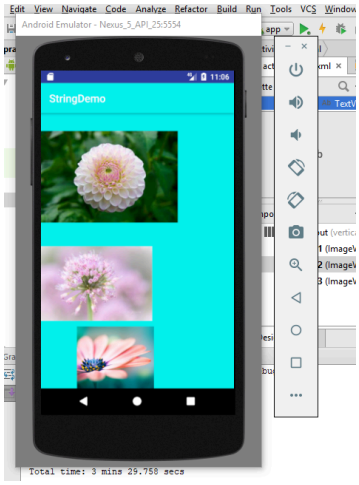
Copy and Paste the three images in the drawable folder.

Activity_main.xml code:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="com.example.student.pract1.MainActivity">
<ImageView
    android:layout_width="256dp"
```

```
    android:layout_height="237dp"
    android:id="@+id/image1"
    android:src="@drawable/image1"/>
<ImageView
        android:id="@+id/image2"
        android:layout_width="208dp"
        android:layout_height="160dp"
        android:src="@drawable/image2" />
<ImageView
        android:id="@+id/image3"
        android:layout_width="279dp"
        android:layout_height="wrap_content"
        android:src="@drawable/image3" />
</LinearLayout>
```

Output:- Drawable



5) Dimension

Dimens.xml

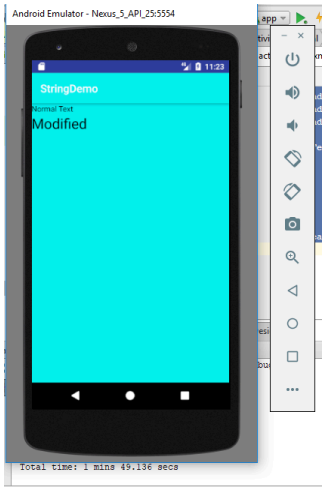
```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<dimen name="textview_height">35dp</dimen>
<dimen name="textview_width">150dp</dimen>
<dimen name="font_size">26sp</dimen>
</resources>
```

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
    tools:context="com.example.student.pract1.MainActivity">
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Normal Text"/>
<TextView
        android:layout_width="@dimen/textview_width"
        android:layout_height="@dimen/textview_height"
        android:textSize="@dimen/font_size"
        android:text="Modified dimension Text"
/>
</LinearLayout>
```

Output:-



Index

Sr No.	Practical Name	DOP	DOS	Sign

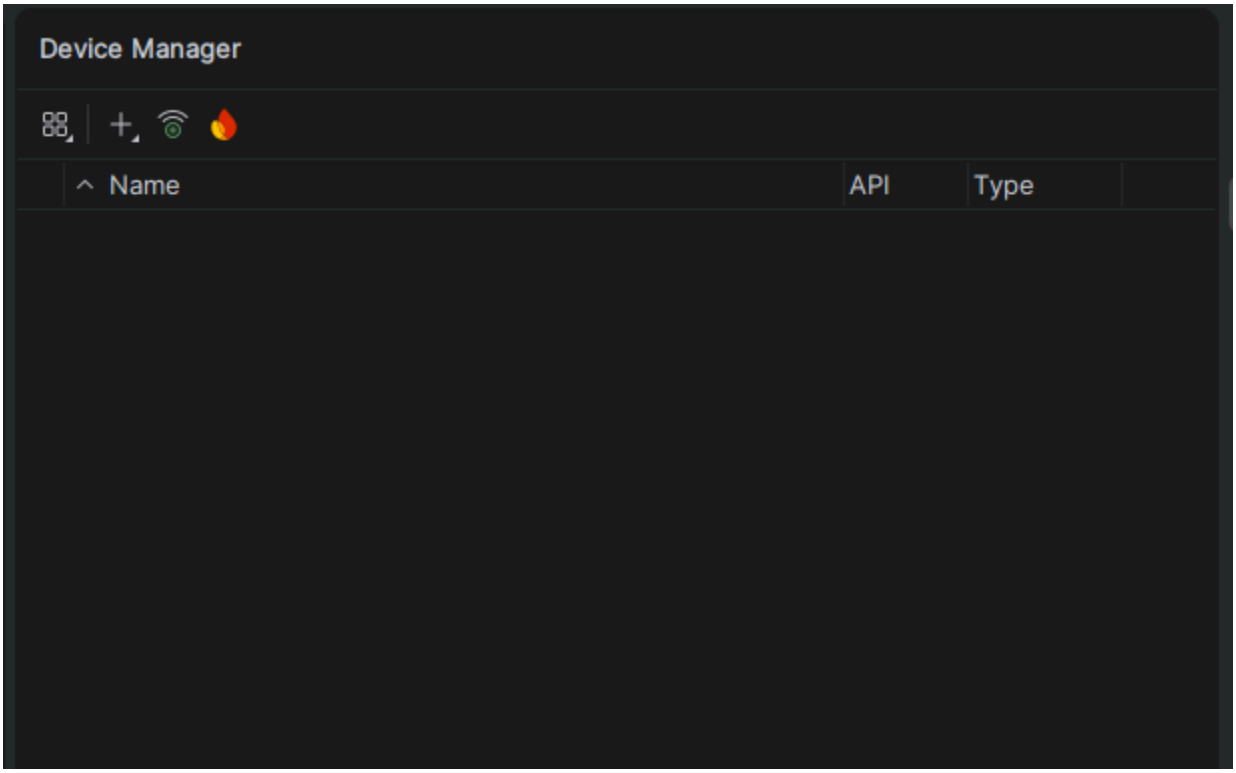
Practical No.1

Aim: Develop a simple “Hello World” Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.

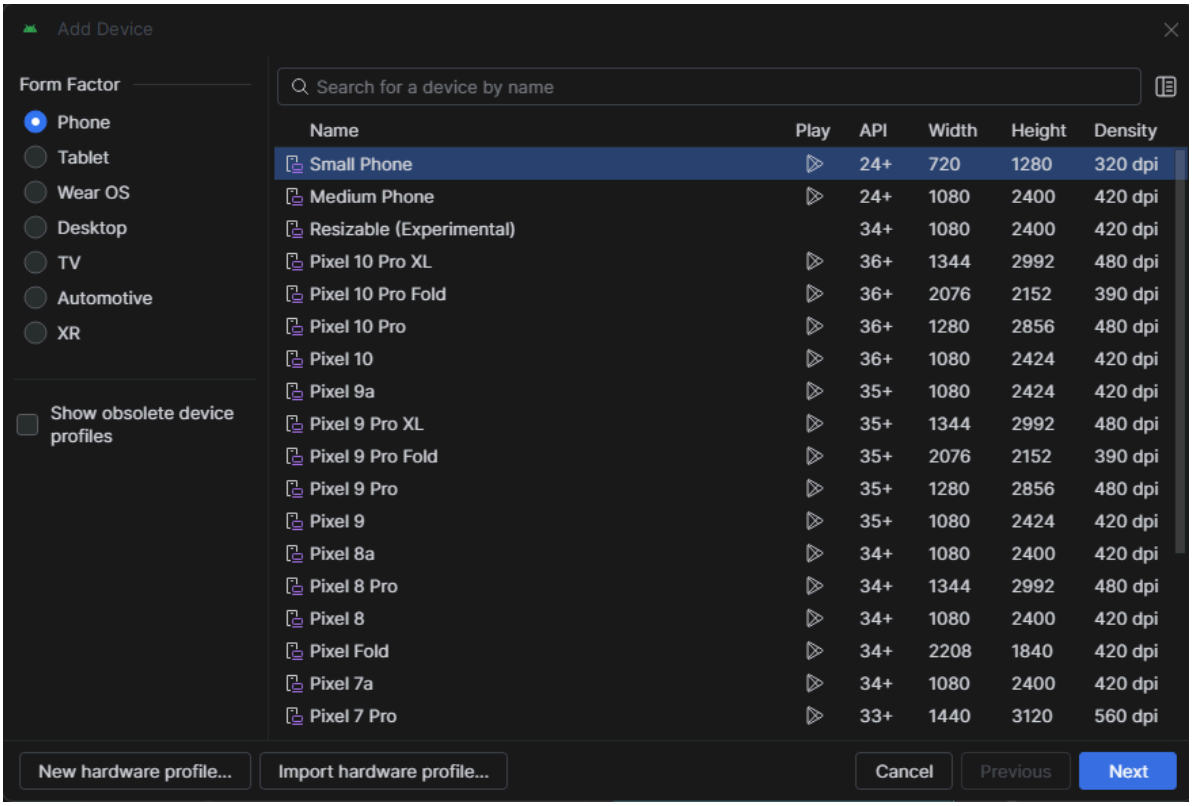
Steps to create Virtual Android Device(AVD)

To open the DeviceManager,Perform following steps:

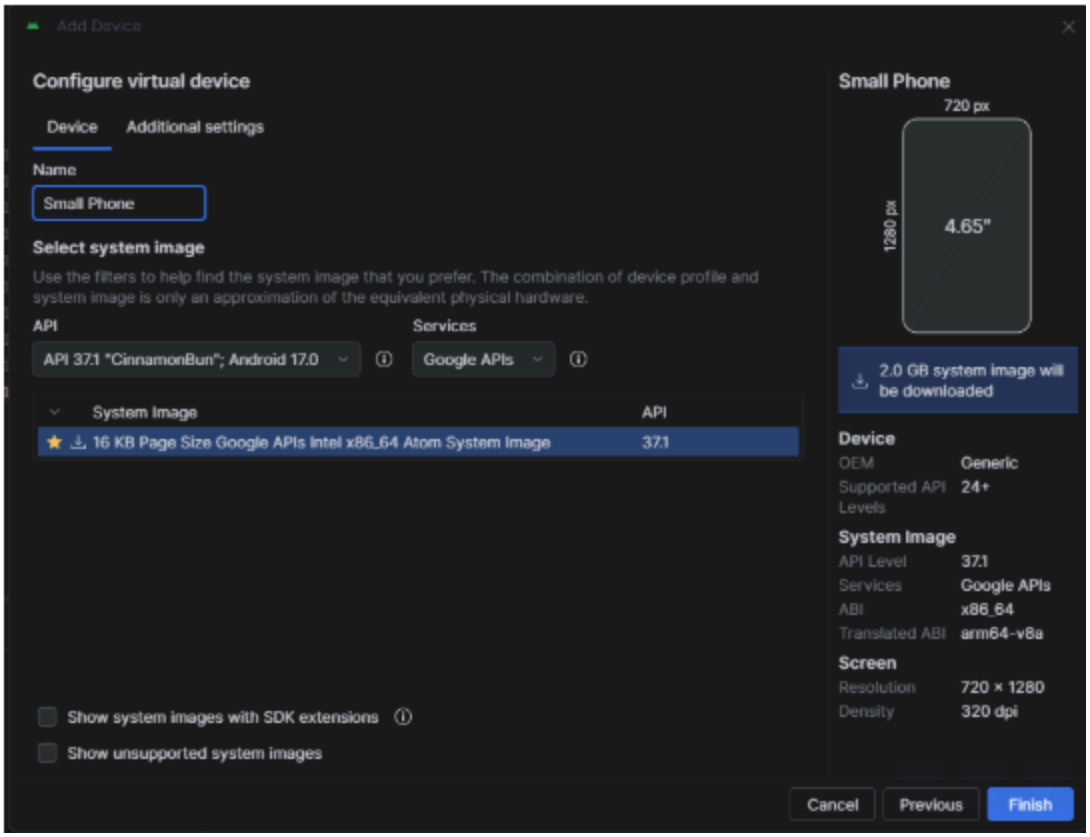
Open the Device Manager by clicking Tools > Device Manager.



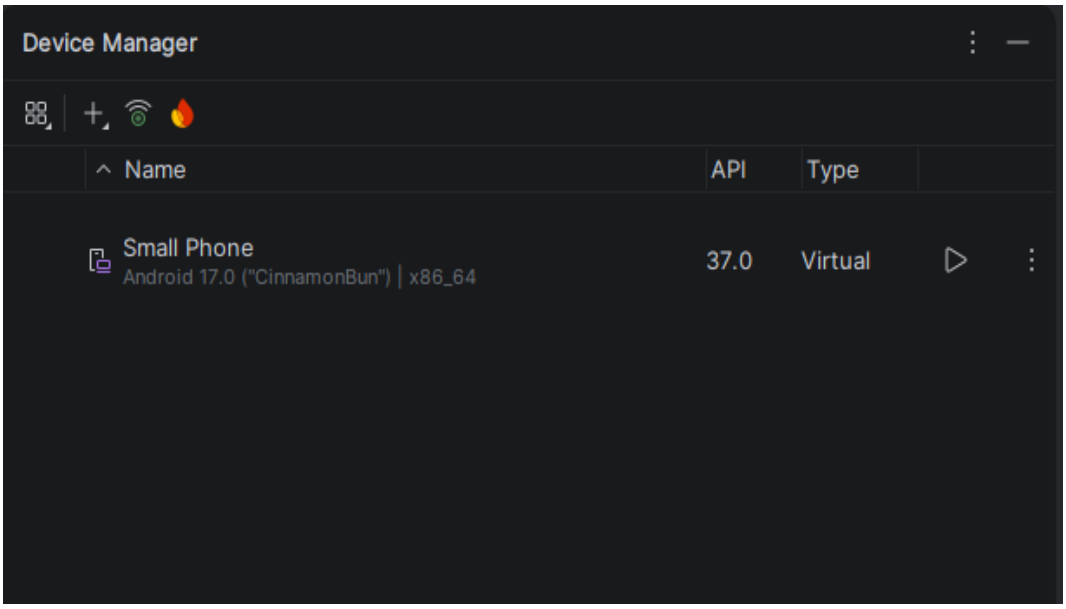
Click on + icon (Create Virtual Device)



Select Form Factor-Phone and select Small Phone as shown in above screen shot and click on Next.



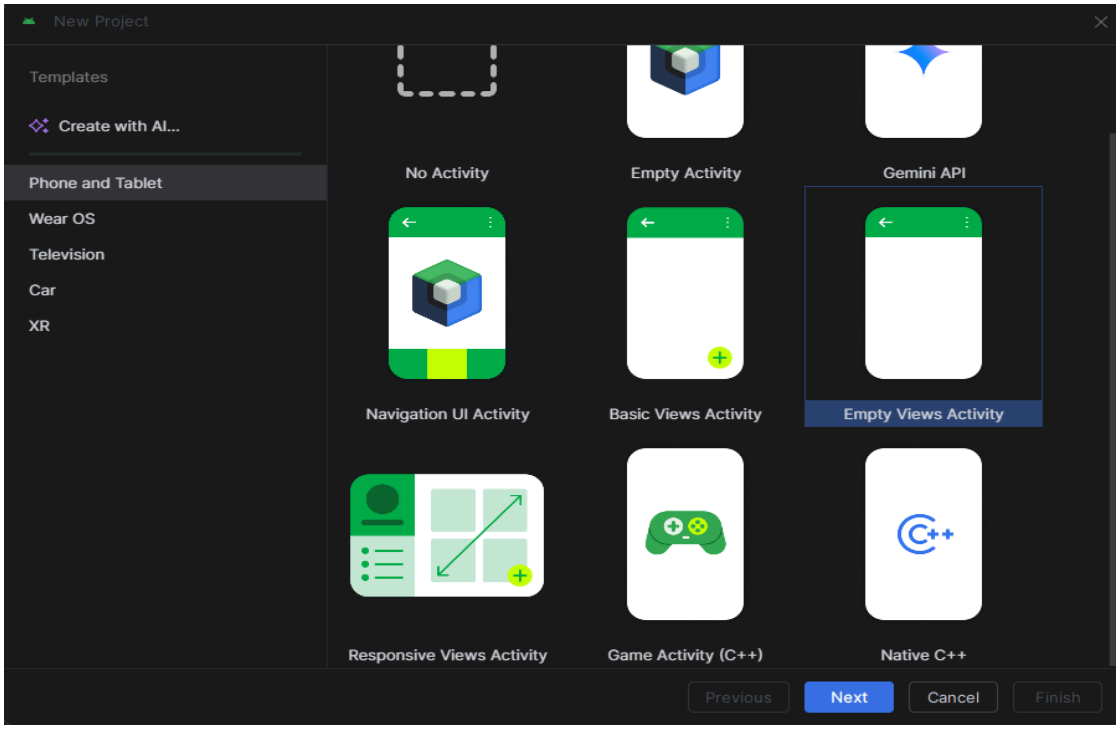
Now Click on Finish



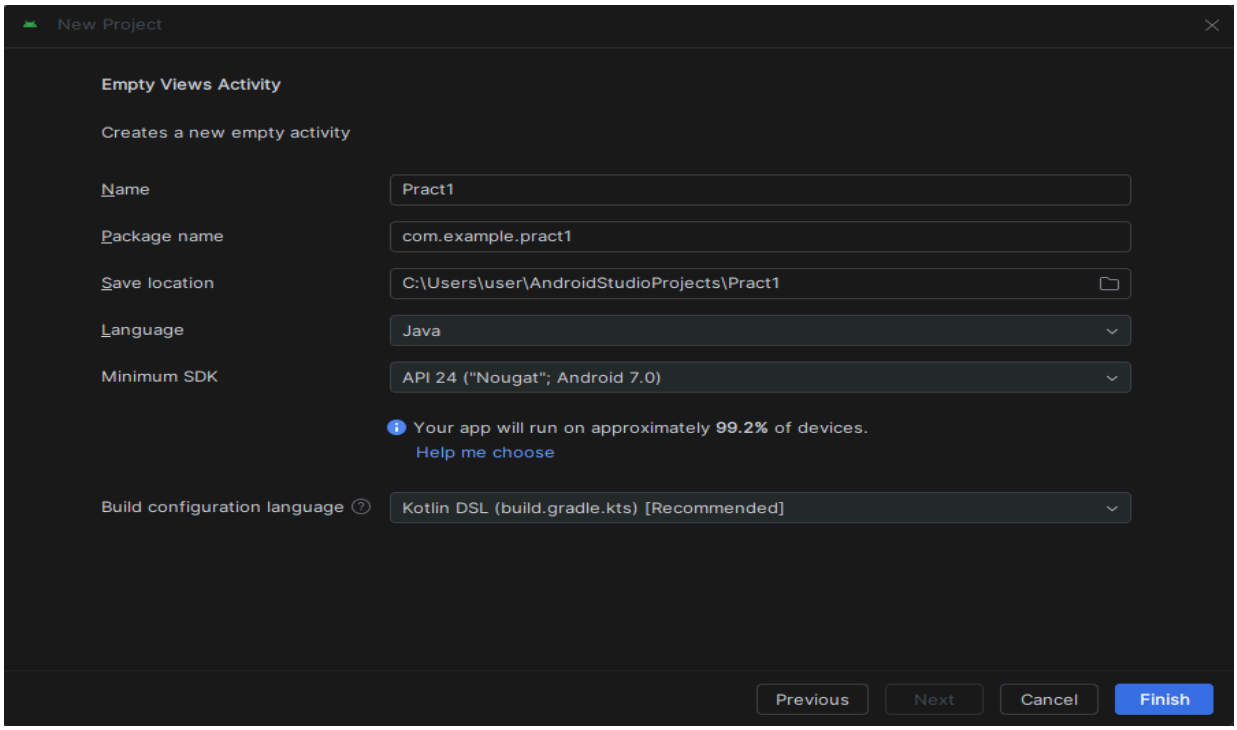
AVD is created now.

Creating simple Hello world Android application

Click on file - new-new project then select Empty view activity-click on Next as shown in the below screenshot



Set the name of your project and make sure java language is selected as shown in the below screenshot.



Keep activity_main.xml and MainActivity.java file as it is.

To build project go to Build - Generate App bundles or APKs - Generate APKs

To Run project Go to -Run-Run App

